ASSIGNMENT 1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Subject: Object Oriented Programming using C++

CO1: Apply object-oriented programming principles and C++ constructs (data types, operators and control structures) to design and write structured OOP-based programs

Total Marks: 50 Nature: Individual Bloom's Level: K5 – Evaluate

Student Name	NELLEPALLI VINAY YADAV
Register Number	192472061
Department / Section	CSE-AI
Submission Date	11/11/2026
GitHub Repository Link	https://github.com/VINAYYADAV10/DSA0117---OBJECT-ORIENTED-PROGRAMMING-WITH-C-
Compiler used	g++ (Ubuntu 13.3.0) 13.3.0
Compile command	g++ -std=c++17 -Wall -Wextra file.cpp -o file

Contents of this report: Task 1 to Task 12 (analysis, alternative concept and complexity analysis, pseudocode, UML, implementation code, execution screenshots and test tables).

Scenario and Common Assumptions

A city metro divides its network into three zones: Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). A stored-value smart card is used and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession. The first version of the gate software had four faults: fares off by a paisa, the peak check treating 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance on a card, and fare values typed in several places.

Assumptions made in this report

- **Money** is stored as an integer number of paise (Rs 1 = 100 paise). Rs 10 = 1000, Rs 20 = 2000, Rs 30 = 3000, Rs 50 = 5000 paise.
- **Peak hours** are the half-open intervals [08:00, 10:00) and [17:00, 20:00). So 08:00 and 17:00 are peak, while 10:00 and 20:00 are off-peak. The time of entry decides the peak status.
- **Concession** is not quantified in the scenario, so a 50% concession is assumed for students and senior citizens, applied after the peak surcharge. It is one constant and can be changed in one place.
- **Minimum balance rule:** balance after deducting the fare must be at least 5000 paise; otherwise the journey is denied and the balance is unchanged.
- **Design challenge (Task 12):** weekend discount (10%) applies only to off-peak journeys; the peak surcharge and the weekend discount never combine, and when both could apply the peak surcharge wins. A monthly pass gives unlimited journeys in two chosen zones.
- **Journey volume:** 20,000 (station), 500,000 (line) and 3,000,000 (network) journeys per day.

Pseudocode convention

All pseudocode in this report uses a structured style: keywords in capitals (ALGORITHM, INPUT, OUTPUT, IF … THEN … ELSE … ENDIF, SWITCH … ENDSWITCH, FOR … ENDFOR, RETURN), the assignment arrow ←, indentation for nesting, and one statement per line. Pseudocode comes before the C++ code for the algorithmic tasks.

How the programs were executed

Every program was compiled with g++ -std=c++17 -Wall -Wextra and executed. The screenshots in this report are the captured compile-and-run sessions (the command line is shown after the $ prompt). Source files are kept one folder per task: Task01 … Task12.

Task 1 – Understand the Problem and Identify the Objects (4 Marks)

1.1 Objects in the problem, their data and behaviour

Object	Data (attributes)	Behaviour (methods)	Why it is an object
SmartCard	card id, balance (long long, paise)	canTravel(fare), deduct(fare), recharge(amount), balance()	It owns the money and the rule that Rs 50 must remain. Having one owner stops any other code from breaking the rule.
Passenger	name, category (general / student / senior)	isEligibleForConcession(), owns a card	Concession depends on who travels, not on the card or the route.
Journey	entry zone, exit zone, entry hour and minute	farthestZone(), isPeak()	A journey is one event with its own time and route; the fare is computed from it.
Zone	Inner, Middle, Outer (enum class)	ordering (Inner < Middle < Outer)	A closed set of named values; an enumeration prevents invalid zone numbers.
FareTable	base fares {1000, 2000, 3000}, surcharge 20%, concession 50%	fare(zone, peak, concession)	All fare values live in one object, so a fare revision is made in one place only.

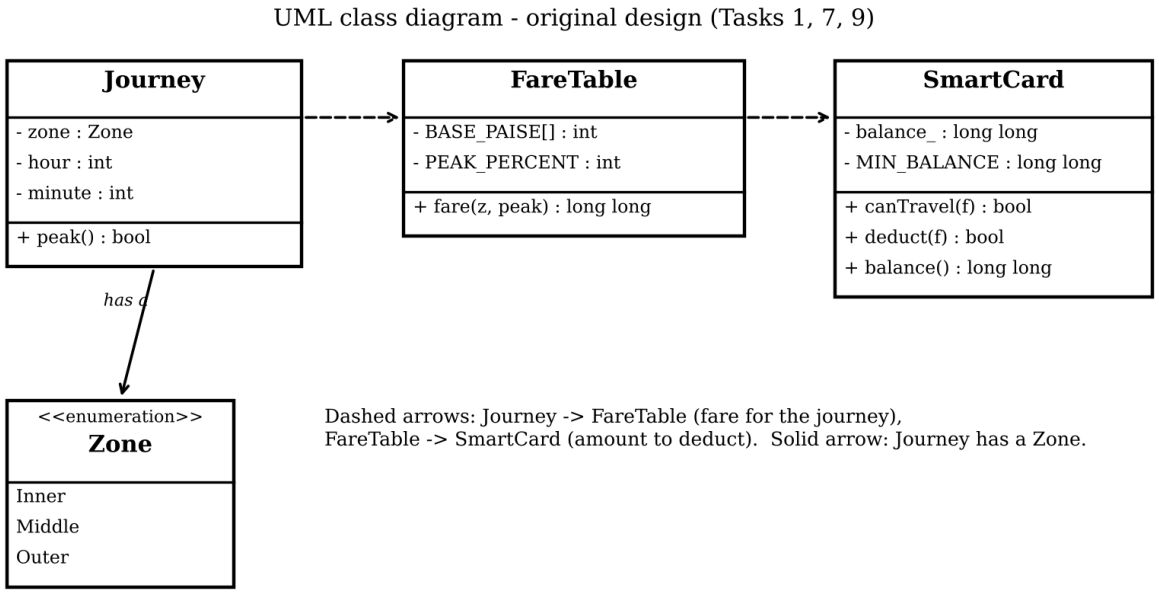


Figure 1.1: UML class diagram of the original design

1.2 Procedural style versus object-oriented style

In a procedural style the problem is a list of steps acting on shared data: a global variable balance, global fare constants, and free functions fare(), canTravel() and deduct(). The data and the rules that protect it are separated, so any function can write balance directly; the minimum-balance rule exists only if every caller remembers to call canTravel() first. If a fare rule changes, every function that has its own copy of the rule must be found and edited.

In an object-oriented style the data and the functions that guard it are bundled in a class. SmartCard keeps balance_ private and exposes canTravel() and deduct(); deduct() checks the rule itself, so the rule cannot be bypassed. FareTable holds the fare values once. The program reads like the problem: a Journey is priced by a FareTable and charged to a SmartCard.

1.3 Basic principles of OOP and where each appears

Principle	Where it appears in this system
Encapsulation	SmartCard::balance_ is private. Only deduct(), recharge() and canTravel() can change or test it, so the balance can never become negative or fall below Rs 50 through a journey.
Abstraction	The gate calls card.deduct(fare) and does not know how the rule is checked. Likewise fare(zone, peak) hides the surcharge arithmetic.
Inheritance	A base class Passenger can be specialised as StudentPassenger and SeniorPassenger; a base class Ticket can be specialised as StoredValueCard and MonthlyPass (Task 12) sharing a common interface.

Principle	Where it appears in this system
Polymorphism	A FarePolicy interface with a virtual function adjust(fare) can be implemented by PeakSurcharge, WeekendDiscount and Concession; the gate applies a list of policies without knowing which are active (run-time polymorphism). Function overloading of fare() is compile-time polymorphism.

1.4 Rules every correct journey must satisfy

No.	Rule	Justification
R1	Balance after the journey ≥ 5000 paise (Rs 50)	Stated business rule; checked before deducting.
R2	Balance is never negative	Follows from R1 and from rejecting negative recharges; the first software broke it.
R3	Fare = base fare of the farthest zone (1000 / 2000 / 3000)	Fare depends on the farthest zone only, not the number of zones.
R4	Peak surcharge of 20% applies in [08:00,10:00) and [17:00,20:00)	Half-open intervals remove the 10 AM / 8 PM inconsistency.
R5	Concession applies for students and senior citizens	Stated business rule.
R6	All money arithmetic is exact (integer paise)	Fixes the "off by a paisa" fault.
R7	Each fare value is defined once	Fixes the "values typed in several places" fault.
R8	A denied journey leaves the balance unchanged	The card must not lose money for a journey it did not allow.
R9	A recharge amount must be positive	Prevents a negative recharge from reducing the balance.

1.5 Implementation (skeleton showing the objects) and execution

File: Task01 / main.cpp

```
// Task 1: objects identified from the Namma Metro fare problem (skeleton + demo)
#include <iostream>

enum class Zone { Inner, Middle, Outer };

class FareTable {                    // knowledge: base fares (single place)
public:
   static constexpr int BASE_PAISE[] = {1000, 2000, 3000};
   static constexpr int PEAK_PERCENT = 20;
   long long fare(Zone farthest, bool peak) const {
      long long f = BASE_PAISE[static_cast<int>(farthest)];
```

```
        if (peak) f += f * PEAK_PERCENT / 100;
        return f;
    }
};

class SmartCard {                   // encapsulation: balance is private
    long long balance_;             // paise
public:
    static constexpr long long MIN_BALANCE = 5000;
    explicit SmartCard(long long b) : balance_(b < 0 ? 0 : b) {}
    bool canTravel(long long f) const { return f >= 0 && balance_ >= MIN_BALANCE
&& f <= balance_ - MIN_BALANCE; }
    bool deduct(long long f) {
        if (!canTravel(f)) return false;
        balance_ -= f;
        return true;
    }
    long long balance() const { return balance_; }
};

class Journey {                     // one trip: a zone, a time, a fare
public:
    Zone farthest; int hour; int minute;
    Journey(Zone z, int h, int m) : farthest(z), hour(h), minute(m) {}
    bool peak() const {
        int t = hour * 60 + minute;
        return (t >= 8 * 60 && t < 10 * 60) || (t >= 17 * 60 && t < 20 * 60);
    }
};

int main() {
    FareTable table;
    SmartCard card(10000);                      // Rs 100.00
    Journey j(Zone::Outer, 18, 30);
    long long f = table.fare(j.farthest, j.peak());
    std::cout << "Fare (paise): " << f << '\n';
    std::cout << "Journey allowed: " << std::boolalpha << card.deduct(f) << '\n';
    std::cout << "Balance (paise): " << card.balance() << '\n';
    std::cout << "Can travel again with Rs 20 fare: " << card.canTravel(2000) << '\n';
    std::cout << "sizeof(SmartCard) = " << sizeof(SmartCard) << " bytes, sizeof(long long)
= " << sizeof(long long) << " bytes\n";
}
```

Terminal - Task 1 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra main.cpp -o main
$ ./main
Fare (paise): 3600
Journey allowed: true
Balance (paise): 6400
Can travel again with Rs 20 fare: false
sizeof(SmartCard) = 8 bytes, sizeof(long long) = 8 bytes
```

Screenshot 1: compile and run of Task01/main.cpp

Result: Outer zone at 18:30 is a peak journey, fare = 3000 + 20% = 3600 paise (Rs 36.00). Balance 10000 → 6400; a further Rs 20 journey is refused because 6400 − 2000 = 4400 < 5000. sizeof(SmartCard) is 8 bytes, equal to its one long long member, so the class adds no memory overhead.

1.6 Alternative Concept and Complexity Analysis – purely procedural solution

The procedural solution would use a global long long balance, a global array of fares and free functions fare(int zone, bool peak), canTravel(long long f) and deduct(long long f). The Task 7 programs implement exactly this alternative.

Measure	**Object-oriented (SmartCard, Journey, FareTable)**	**Procedural (globals + free functions)**
Time per journey	O(1): one table lookup, a few comparisons, one subtraction	O(1): same operations
Memory per card	O(1): sizeof(SmartCard) = 8 bytes (measured)	O(1): one 8-byte balance per card (array of long long for many cards)
Code changed when one fare rule changes	One place: the FareTable / constant; all callers unchanged	Every function or file that duplicates the rule, plus every caller of deduct() to add checks
Protection of the Rs 50 rule	Enforced inside deduct()	Depends on every caller; Task 7 shows it is broken

Conclusion: time and space are the same in Big-O terms (and no virtual functions are used, so there is no run-time cost); the object-oriented style wins on maintainability and correctness.

Task 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

2.1 The five kinds of tokens (examples from Task02/main.cpp)

Token kind	Meaning	Examples from the program
Keywords	Reserved words with fixed meaning	int, long, const, constexpr, enum, class, static_cast, return, if, switch
Identifiers	Names chosen by the programmer	BASE_PAISE, fare, Zone, MIN_BALANCE_PAISE, x, y
Constants (literals)	Fixed values	1000, 2000, 3000, 5000, 20, 'A', "Middle fare (paise) = "
Operators	Symbols that perform operations	=, +, <<, ==, -, ::, [ ]
Punctuators	Separators and delimiters	; , { } ( ) [ ] #

Example line: int fare = BASE_PAISE[static_cast<int>(Zone::Middle)]; has the tokens: int (keyword), fare (identifier), = (operator), BASE_PAISE (identifier), [(punctuator), static_cast (keyword), < (operator), int (keyword), > (operator), ((punctuator), Zone (identifier), :: (operator), Middle (identifier),) (punctuator),] (punctuator), ; (punctuator).

2.2 #define, const and constexpr – and why constexpr is preferred for the fare table

Feature	#define	const	constexpr
Handled by	Pre-processor (text replacement)	Compiler	Compiler (compile-time)
Type	None	Yes	Yes
Scope	Ignores scope, global from the point of definition	Follows block / namespace scope	Follows block / namespace scope
Value known at compile time	Yes (literal text)	Only if initialised by a constant expression	Always guaranteed (error otherwise)
Usable as array size / switch case	Yes	Sometimes	Yes, guaranteed
Debugger / error messages	Shows the replaced number, no name	Shows the name	Shows the name

constexpr is preferred for the fare table because the fares are known before the program runs: the compiler checks the type, rejects any attempt to change them, can use them in array sizes and static_assert, and can fold them into the code. A #define would have no type or scope, and a plain const may be a run-time value.

2.3 enum and enum class for zones – why enum class is safer

A plain enum (enum OldZone { INNER, MIDDLE, OUTER }) puts its names in the surrounding scope, so INNER could clash with another name, and its values convert silently to int, so int x = OUTER and comparisons such as x == 2 compile without complaint. An enum class (enum class Zone { Inner, Middle, Outer }) keeps its names inside Zone::, has no implicit conversion to or from int (Zone z = 1 and int i = Zone::Inner are errors), and therefore cannot be mixed up with an ordinary number such as an hour. A static_cast<int> is needed when the zone is used as an array index, which makes the conversion visible.

2.4 Naming rules and conventions for identifiers

- **Rules (enforced by the compiler):** an identifier may contain letters, digits and underscore; it must not start with a digit; it must not be a keyword; it is case-sensitive; no spaces or symbols.
- **Avoid reserved forms:** names starting with an underscore followed by a capital letter, or containing a double underscore, are reserved for the implementation.
- **Conventions used here:** UPPER_SNAKE_CASE for constants (BASE_PAISE, MIN_BALANCE_PAISE); PascalCase for types and enumerators (SmartCard, Zone::Inner); camelCase for functions and variables (canTravel, fareInPaise); a trailing underscore for private data members (balance_); meaningful names with the unit in the name (_PAISE, _PERCENT).

2.5 Implementation and execution

File: Task02 / main.cpp

```
// Task 2: tokens, keywords, identifiers and constants - one place for all fare values
#include <iostream>

#define OLD_SURCHARGE 20                    // macro: no type, no scope
constexpr int PEAK_SURCHARGE_PERCENT = 20;     // typed, scoped compile-time
constant
constexpr long long MIN_BALANCE_PAISE = 5000;
const int RUNTIME_LIMIT = static_cast<int>(MIN_BALANCE_PAISE) + 0;  // const
(may be runtime)

enum class Zone { Inner, Middle, Outer };      // scoped enumeration
enum OldZone { INNER, MIDDLE, OUTER };         // plain enum leaks names
constexpr int BASE_PAISE[] = {1000, 2000, 3000};

int main() {
   int fare = BASE_PAISE[static_cast<int>(Zone::Middle)];   // identifier, constant,
punctuators
   std::cout << "Tokens in the line above: keyword(int) identifier(fare) operator(=) "
             "constant(1000) punctuator(;)\n";
   std::cout << "constexpr surcharge = " << PEAK_SURCHARGE_PERCENT
          << ", macro surcharge = " << OLD_SURCHARGE << '\n';
   std::cout << "Middle fare (paise) = " << fare << '\n';
```

```
    int x = INNER;              // plain enum converts silently to int
    int y = OUTER;
    std::cout << "Plain enum: INNER == 0 ? " << (x == 0) << ", INNER == OUTER-2 ? "
<< (x == y - 2) << '\n';
    // int bad = Zone::Inner;   // ERROR with enum class: no implicit conversion
    // Zone z = 1;              // ERROR: int is not a Zone
    Zone z = Zone::Outer;
    std::cout << "enum class needs a cast: " << static_cast<int>(z) << '\n';
    std::cout << "sizeof(Zone)=" << sizeof(Zone) << " sizeof(BASE_PAISE)=" <<
sizeof(BASE_PAISE)
              << " RUNTIME_LIMIT=" << RUNTIME_LIMIT << '\n';
}
```

Terminal - Task 2 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra main.cpp -o main
$ ./main
Tokens in the line above: keyword(int) identifier(fare) operator(=) constant(1000) punctuator(;)
constexpr surcharge = 20, macro surcharge = 20
Middle fare (paise) = 2000
Plain enum: INNER == 0 ? 1, INNER == OUTER-2 ? 1
enum class needs a cast: 2
sizeof(Zone)=4 sizeof(BASE_PAISE)=12 RUNTIME_LIMIT=5000
```

Screenshot 2: compile and run of Task02/main.cpp

Result: the constexpr and macro surcharge both give 20, so the macro gives no benefit but loses type and scope. The plain enum value INNER silently equals 0 and takes part in arithmetic (INNER == OUTER-2 is true), which an enum class forbids; the lines int bad = Zone::Inner; and Zone z = 1; are shown commented out because they are compile-time errors. sizeof(BASE_PAISE) = 12 bytes (three 4-byte ints).

A single, well-named set of constants is recommended for the final program: BASE_PAISE, PEAK_SURCHARGE_PERCENT, CONCESSION_PERCENT, MIN_BALANCE_PAISE (all constexpr) and enum class Zone.

2.6 Alternative Concept and Complexity Analysis – #define macros instead of constexpr and enum class

With macros the program would be #define BASE_INNER 1000, #define BASE_MIDDLE 2000, #define BASE_OUTER 3000, #define PEAK_PCT 20, and zones would be #define INNER 0 and so on.

Aspect	#define macros	constexpr + enum class
Type checking	None: the preprocessor pastes text; a wrong type is found only after substitution or never	Full type checking; Zone cannot be assigned an int

Aspect	#define macros	constexpr + enum class
Scope	None: macro names are global and can silently replace identifiers elsewhere (e.g. a variable named INNER)	Namespace / class scope; Zone::Inner cannot clash
Pitfalls	Operator precedence surprises, e.g. #define TWICE(x) x*2 expands TWICE(3+1) to 3+1*2 = 5, not 8	None of these
Memory	No storage: the literal is inside the instructions	A constexpr value used only in expressions also needs no storage; an array used with a variable index occupies 12 bytes of read-only data (measured sizeof)
Running time	O(1)	O(1); the compiler folds constants, so there is no run-time difference

Task 3 – Choose Data Types and Check Type Compatibility (4 Marks)

3.1 Type chosen for every value (sizes measured on this system)

Value	Type chosen	sizeof	Reason
Fare, balance, daily collection (paise)	long long	8	Exact whole numbers; the network total reaches 10.8 billion paise, beyond int
Base fare table entries	int (or long long)	4	Each entry ≤ 3000, safely inside int
Peak flag, concession flag	bool	1	Two-valued conditions
Concession code (G/S/R)	char	1	A single character is enough
Hour and minute	int	4	Small range 0–23, 0–59; minutes since midnight 0–1439 also fits
Zone number	enum class Zone (underlying int)	4	Named values; invalid numbers are not possible
Journey count	unsigned (or long long)	4	Never negative; 3,000,000 per day fits easily (max 4,294,967,295)
Percentages (20, 50)	int	4	Whole numbers; integer arithmetic is exact
Display-only averages	double	8	Only for display, never for stored money

3.2 Why money is stored in integer paise and not in double rupees

Binary floating point cannot represent most decimal fractions exactly. 0.10 and 0.20 are stored as slightly wrong binary values, so 0.1 + 0.2 == 0.3 is false and many additions accumulate an error. Integers are exact, the comparison balance − fare >= 5000 is exact, and the arithmetic cannot produce a fare that is off by a paisa. Rounding is then a deliberate, visible decision (integer division) and not an accident.

3.3 Type compatibility rules and mixed expressions

- **Integral promotion:** bool, char and short are promoted to int before arithmetic (so 'A' + 2 is an int, true + true is 2).
- **Usual arithmetic conversions:** if one operand is double the other is converted to double; otherwise if one is unsigned and the same rank as the other signed one, the signed value is converted to unsigned; otherwise the smaller type is widened.
- **bool conversions:** zero converts to false, any non-zero value converts to true; true converts to 1 and false to 0.

- **Result:** int + double → double; int + bool → int; int comparisons with unsigned → unsigned comparison (a trap, see Task 4).

3.4 Ranges and whether int is enough for a day's collection

INT_MAX = 2,147,483,647 paise = about Rs 21.4 million. The worst-case network day is 3,000,000 journeys × Rs 36.00 = 10,800,000,000 paise, which is more than five times INT_MAX. int is therefore not enough for a network total; long long (maximum about 9.22 × 10^18) is.

3.5 Implementation and execution

File: Task03 / main.cpp

```
// Task 3: data types, sizeof, limits, double-vs-integer money, overflow
#include <climits>
#include <cfloat>
#include <iomanip>
#include <iostream>

int main() {
   std::cout << std::left << std::setw(26) << "Type (use)" << std::setw(8) << "sizeof" <<
"Range\n";
   std::cout << std::setw(26) << "bool (peak flag)" << std::setw(8) << sizeof(bool) <<
"false/true\n";
   std::cout << std::setw(26) << "char (concession code)" << std::setw(8) << sizeof(char)
<< int(CHAR_MIN) << " to " << int(CHAR_MAX) << '\n';
   std::cout << std::setw(26) << "int (zone no., hour)" << std::setw(8) << sizeof(int) <<
INT_MIN << " to " << INT_MAX << '\n';
   std::cout << std::setw(26) << "unsigned (journey count)" << std::setw(8) <<
sizeof(unsigned) << "0 to " << UINT_MAX << '\n';
   std::cout << std::setw(26) << "long long (paise)" << std::setw(8) << sizeof(long long)
<< LLONG_MIN << " to " << LLONG_MAX << '\n';
   std::cout << std::setw(26) << "float" << std::setw(8) << sizeof(float) << "+-" <<
FLT_MAX << " (~7 digits)\n";
   std::cout << std::setw(26) << "double (not for money)" << std::setw(8) <<
sizeof(double) << "+-" << DBL_MAX << " (~15 digits)\n";

   std::cout << "\n-- Precision problem: add Rs 0.10 one million times --\n";
   double rupees = 0.0; long long paise = 0;
   for (int i = 0; i < 1000000; ++i) { rupees += 0.10; paise += 10; }
   std::cout << std::fixed << std::setprecision(10);
   std::cout << "double rupees : " << rupees << "\n";
   std::cout << "integer paise : " << paise / 100 << '.' << std::setw(2) << std::setfill('0') <<
paise % 100 << std::setfill(' ') << "\n";
   std::cout << "error (double): " << rupees - 100000.0 << "\n";
   std::cout << "0.1+0.2==0.3 ? " << std::boolalpha << (0.1 + 0.2 == 0.3) << "\n";

   std::cout << "\n-- Growth of rounding error with n fares of Rs 0.10 --\n";
   for (long long n : {1000LL, 10000LL, 100000LL, 1000000LL, 10000000LL,
100000000LL}) {
```

```
        double s = 0; for (long long i = 0; i < n; ++i) s += 0.10;
        std::cout << std::setw(10) << n << "  error = " << std::scientific << std::setprecision(3)
<< s - n * 0.10 << "\n";
    }

    std::cout << std::fixed << "\n-- Does int hold a day's network collection? --\n";
    long long network = 3000000LL * 3600;   // worst case: 3,000,000 journeys at Rs 36.00
(paise)
    std::cout << "worst-case daily total (paise) = " << network << "\n";
    std::cout << "INT_MAX                        = " << INT_MAX << "\n";
    std::cout << "fits in int ? " << (network <= INT_MAX) << "  fits in long long ? " <<
(network <= LLONG_MAX) << "\n";
}
```

Terminal - Task 3 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra main.cpp -o main
$ ./main
Type (use)                sizeof  Range
bool (peak flag)          1       false/true
char (concession code)    1       -128 to 127
int (zone no., hour)      4       -2147483648 to 2147483647
unsigned (journey count)  4       0 to 4294967295
long long (paise)         8       -9223372036854775808 to 9223372036854775807
float                     4       +-3.40282e+38 (~7 digits)
double (not for money)    8       +-1.79769e+308 (~15 digits)

-- Precision problem: add Rs 0.10 one million times --
double rupees : 100000.0000013329
integer paise : 100000.00
error (double): 0.0000013329
0.1+0.2==0.3 ? false

-- Growth of rounding error with n fares of Rs 0.10 --
1000        error = -1.407e-12
10000       error = 1.588e-10
100000      error = 1.885e-08
1000000     error = 1.333e-06
10000000    error = -1.610e-04
100000000   error = -1.887e-02

-- Does int hold a day's network collection? --
worst-case daily total (paise) = 10800000000
INT_MAX                        = 2147483647
fits in int ? false  fits in long long ? true
```

Screenshot 3: compile and run of Task03/main.cpp (sizes, ranges, precision and overflow evidence)

3.6 Evidence of a precision problem and its resolution

Experiment	double rupees	integer paise
Add Rs 0.10 one million times (true answer Rs 100000.00)	100000.0000013329 (error 1.333 × 10^-6)	100000.00 exactly

Experiment	double rupees	integer paise
0.1 + 0.2 == 0.3	false	not applicable (10 + 20 == 30 is true)
Daily total 10,800,000,000 paise in int	Overflow: does not fit in int (fits long long)	Resolved by using long long

Resolution: store paise in long long. The precision problem disappears and the overflow problem is solved by the wider type.

3.7 Alternative Concept and Complexity Analysis – money as double rupees

Solution with double rupees: double balance = 100.00; fare = base * 1.2 for peak; check balance - fare >= 50.0 (needs an epsilon such as 1e-9 to be safe, e.g. balance - fare + 1e-9 >= 50.0). Display would use setprecision(2) and rounding.

Measure	long long paise	double rupees
Time per addition	O(1); 0.53 ns measured (Task 10, 200 million additions)	O(1); 2.19 ns measured (floating-point add has a longer latency)
Space per value	8 bytes	8 bytes
Rounding error after n additions of Rs 0.10	Zero	n = 10^3: 1.4e-12; 10^4: 1.6e-10; 10^5: 1.9e-08; 10^6: 1.3e-06; 10^7: 1.6e-04; 10^8: 1.9e-02 (measured)

The measured error grows roughly 100 times for every 10 times more fares, because every addition rounds relative to the (growing) running total and the errors are systematic. The time and space costs are the same in Big-O terms, but only the integer version is exact.

Task 4 – Control Implicit Conversions (4 Marks)

4.1 Places where C++ converts silently

- **Integer division and integer-to-double conversion:** a / b with a = 7 and b = 2 is evaluated in int and gives 3; only then is 3 converted to 3.0 when stored in a double. The fraction is already lost. Writing static_cast<double>(a) / b converts before dividing and gives 3.5.
- **Comparison of signed and unsigned values:** in s < u (int s = -1, unsigned u = 1) the int is converted to unsigned: -1 becomes 4,294,967,295, so the comparison is false. g++ warns with -Wsign-compare.
- **Narrowing conversions:** int x = 3.7; silently truncates to 3. Brace initialisation int x{3.7}; is a compile-time error, as are bool c{5}; and char d{300};. This is the safeguard against accidental data loss (compiler errors are shown in Screenshot 4b).
- **static_cast:** an explicit cast is needed when the narrowing is intended (static_cast<int>(3.7)), when real division is needed (static_cast<double>(a) / b), and when an enum class is used as an index (static_cast<int>(Zone::Outer)).

4.2 Predict the Output

Program (given in the assignment, stored as Task04/predict.cpp):

File: Task04 / predict.cpp

```
#include <iostream>

int main()
{
   int a = 7, b = 2;
   double c = a / b;
   double d = static_cast<double>(a) / b;
   std::cout << c << ' ' << d << '\n';

   char ch = 'A' + 2;
   std::cout << ch << ' ' << ch + 1 << '\n';

   bool flag = 5;
   std::cout << flag << ' ' << std::boolalpha << flag << '\n';

   unsigned int u = 1;
   int s = -1;
   std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

My predicted output (written before running):

```
3 3.5
C 68
1 true
s >= u
```

I also predicted that g++ -Wall -Wextra would warn about the last line (comparison of integer expressions of different signedness).

Terminal - Task 4 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra predict.cpp -o predict
predict.cpp: In function 'int main()':
predict.cpp:18:21: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]
   18 |     std::cout << (s < u ? "s < u" : "s >= u") << '\n';
      |                   ~~^~~
$ ./predict
3 3.5
C 68
1 true
s >= u
$ g++ -std=c++17 -Wall -Wextra corrected.cpp -o corrected
$ ./corrected
3.5 3 C 1 true
$ g++ -std=c++17 -Wall -Wextra narrowing_demo.cpp -o narrowing_demo
narrowing_demo.cpp: In function 'int main()':
narrowing_demo.cpp:4:11: error: narrowing conversion of '3.7000000000000002e+0' from 'double' to 'int' [-Wnarrowing]
    4 |     int b{3.7};         // ERROR: narrowing double -> int
      |           ^~~
narrowing_demo.cpp:5:12: error: narrowing conversion of '5' from 'int' to 'bool' [-Wnarrowing]
    5 |     bool c{5};          // ERROR: narrowing int -> bool
      |            ^
narrowing_demo.cpp:6:12: error: narrowing conversion of '300' from 'int' to 'char' [-Wnarrowing]
    6 |     char d{300};        // ERROR: constant 300 does not fit in char
      |            ^~~
```

Screenshot 4: compile and run of Task04 (warning, actual output, corrected program, and the narrowing-error demonstration)

4.3 Explanation of each line of the actual output (prediction = actual)

Output line	Why	Conversion that took place
3 3.5	a / b is int / int = 3, then 3 is stored in double c, printed as 3 (cout drops the .0). d divides 7.0 by 2 (b is converted to double), giving 3.5.	Integer division, then int to double (c); int to double before the division (d)
C 68	'A' (65) is promoted to int, 65 + 2 = 67, stored back in char as 'C'. In ch + 1 the char is promoted to int, so the int 68 is printed, not a character.	char to int promotion; int to char on initialisation
1 true	The int 5 is non-zero, so bool flag = 5 stores true. cout prints a bool as 1 by default and as true after std::boolalpha.	int to bool
s >= u	s (-1) is converted to unsigned int 4294967295, which is larger than 1, so s < u is false.	Signed to unsigned (usual arithmetic conversion)

Difference between prediction and actual output: none in the program output. The only extra item was the compiler warning -Wsign-compare for the line (s < u ? ...), meaning: the compiler found an int compared with an unsigned int, a likely logic error.

4.4 Corrected code (brace initialisation and static_cast)

File: Task04 / corrected.cpp

```
// Task 4 (corrected): brace initialisation and static_cast remove the surprises
#include <iostream>

int main() {
   int a{7}, b{2};
   double c{static_cast<double>(a) / b};          // intended real division
   // int x{3.7};                                 // ERROR: narrowing double -> int
   int x = static_cast<int>(3.7);                 // explicit, intended truncation
   char ch{static_cast<char>('A' + 2)};
   bool flag{true};                               // bool flag{5}; would be a narrowing error
   unsigned int u{1};
   int s{-1};
   bool smaller = s < static_cast<int>(u);        // compare as signed on purpose
   std::cout << c << ' ' << x << ' ' << ch << ' ' << flag << ' ' << std::boolalpha << smaller <<
'\n';
}
```

File: Task04 / narrowing_demo.cpp

```
// Brace initialisation rejects narrowing conversions at compile time
int main() {
   int a = 3.7;        // allowed: silently truncates to 3 (no error, maybe a warning)
   int b{3.7};         // ERROR: narrowing double -> int
   bool c{5};          // ERROR: narrowing int -> bool
   char d{300};        // ERROR: constant 300 does not fit in char
   return a + b + c + d;
}
```

Result: the corrected program prints 3.5 3 C 1 true; the signed comparison is now made deliberately as signed (s < static_cast<int>(u) is true). narrowing_demo.cpp is deliberately wrong code: g++ refuses to compile it and reports three [-Wnarrowing] errors (shown in Screenshot 4), proving that brace initialisation catches double→int, int→bool and out-of-range constant→char conversions at compile time.

4.5 Alternative Concept and Complexity Analysis – every conversion explicit

If every conversion were written with static_cast and every variable initialised with braces, the solution would be the code in corrected.cpp applied to the whole program.

- **Running time:** unchanged, O(1) per conversion. static_cast and an implicit conversion generate the same machine instruction (for example cvtsi2sd for int to double); no extra run-time check is added.
- **Memory:** unchanged; casts do not create extra storage.
- **Errors caught at compile time instead of run time:** narrowing in brace initialisation (double→int, int→bool, constant overflow) becomes a compile error; implicit enum class conversions are errors; with -Wconversion and -Wsign-conversion and -Werror the

signed/unsigned comparison also becomes a build failure. With implicit conversions the same mistakes compile, run, and give wrong fares silently.

- **Cost:** more typing and slightly noisier code; static_cast does not itself make a wrong cast safe, so the programmer must still be correct.

Task 5 – Write Correct Expressions with the Right Precedence (4 Marks)

5.1 Expressions for the peak check, the surcharge and the concession

Key expressions

```
bool isPeakHour(int hour) { return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20); }

fare = peak ? base + base * SURCHARGE_PERCENT / 100 : base;

fare = (student || senior) ? fare - fare * CONCESSION_PERCENT / 100 : fare;
```

5.2 Precedence and associativity in each expression

Expression	Order of evaluation
(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)	Relational operators >=, < (higher) first; then && (higher than ||) inside each bracket; then || last. Left-to-right for && and ||, with short-circuiting.
base + base * SURCHARGE_PERCENT / 100	* and / have the same precedence and associate left to right: (base * 20) / 100 first; then + is applied. Multiplying before dividing keeps the arithmetic exact in integers.
peak ? a : b	The conditional operator has very low precedence (above assignment, below ||) and associates right to left. Because << is higher than ?:, a conditional used inside cout << must be in parentheses: cout << (x > y ? x : y).
fare -= fare * 50 / 100 (compound assignment)	Assignment operators have the lowest precedence and associate right to left; the right side is fully evaluated first, then fare is updated.

5.3 Difference between x++ and ++x, and why i = i++ is a mistake

x++ (post-increment) yields the old value of x and then increments it; ++x (pre-increment) increments first and yields the new value. In int z = x++ + ++y; with x = 5, y = 3: x++ yields 5 (x becomes 6) and ++y yields 4 (y becomes 4), so z = 9. i = i++ is a mistake because it both reads i, increments i and assigns to i in one expression: the programmer's intention is unclear, and before C++17 the behaviour was undefined. In C++17 the value 5 is assigned back so the increment is lost. g++ warns with -Wsequence-point (see Screenshot 5). The clear form is simply i++; or i = i + 1;.

5.4 Result of / and % with negative operands

Since C++11 integer division truncates toward zero and the remainder takes the sign of the dividend, so that (a / b) * b + a % b == a always holds. 10 / 3 = 3 and 10 % 3 = 1, but -10 / 3 = -3 and -10 % 3 = -1 (not 2). The unary minus has higher precedence than / and %, so -10 / 3 means (-10) / 3.

5.5 Why the bracketed peak expression is preferred

(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and equivalent to hour >= 8 && hour < 10 || hour >= 17 && hour < 20, because && has higher precedence than ||. But the second form forces the reader to remember that rule; the brackets show each time window as one unit, so the expression reads as “morning window or evening window” and cannot be mis-edited by a maintainer who assumes a different grouping.

5.6 Predict the Output

File: Task05 / predict.cpp

```
#include <iostream>

int main()
{
   int x = 5, y = 3;
   std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

   int z = x++ + ++y;
   std::cout << z << ' ' << x << ' ' << y << '\n';

   std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
   std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

   int k = 0;
   for (int i = 0; i < 10; i += 3) k += i;
   std::cout << k << '\n';
}
```

My predicted output (written before running):

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

```
Terminal - Task 5 (g++ 13, Ubuntu)

$ g++ -std=c++17 -Wall -Wextra predict.cpp -o predict
$ ./predict
11 16
9 6 4
6 4
3 1 -3 -1
18
$ g++ -std=c++17 -Wall -Wextra expressions.cpp -o expressions
expressions.cpp: In function 'int main()':
expressions.cpp:27:7: warning: operation on 'i' may be undefined [-Wsequence-point]
   27 |     i = i++ + 1;                    // BAD: modifies i twice (undefined before C++17 rules; unspecified result style)
      |     ~~^~~~~~~~
expressions.cpp:27:7: warning: operation on 'i' may be undefined [-Wsequence-point]
$ ./expressions
range-check and table agree for all 24 hours: true
hour 7 peak? false
hour 8 peak? true
hour 9 peak? true
hour 10 peak? false
hour 16 peak? false
hour 17 peak? true
hour 19 peak? true
hour 20 peak? false
Outer peak fare = 3600 paise; student = 1800 paise
bad style i=6, clear style j=7
m += 3*2 -> 13
```

Screenshot 5: compile and run of Task05 (predict.cpp and expressions.cpp)

5.7 Line-by-line operator explanation (prediction = actual)

Line	Operators and order	Result
x + y * 2 and (x + y) * 2	* before + (precedence); in the second the parentheses force + first. << (lowest here) sends each result to cout.	5 + 6 = 11 and 8 * 2 = 16
z = x++ + ++y	Postfix ++ and prefix ++ (highest), then binary +, then = (right to left). x++ yields 5, x becomes 6; ++y makes y = 4 and yields 4.	z = 9, x = 6, y = 4
(x > y ? x : y)	Parentheses first; inside, > is evaluated before ?:. x = 6 > y = 4 is true, so the result is x.	6
x % 4 * 2	% and * have equal precedence, left to right: (6 % 4) * 2.	2 * 2 = 4
10 / 3, 10 % 3	Integer division truncates; % gives the remainder.	3 and 1
-10 / 3, -10 % 3	Unary minus first, then / or %; truncation toward zero; remainder takes the sign of -10.	-3 and -1
for (i = 0; i < 10; i += 3) k += i;	+= is compound assignment (right to left). i takes 0, 3, 6, 9.	k = 0 + 3 + 6 + 9 = 18

Difference between prediction and actual output: none. The program compiled with no warnings.

5.8 Complete program for the Task 5 expressions and execution

File: Task05 / expressions.cpp

```
// Task 5: peak check, surcharge and concession expressions
#include <iostream>
```

```
constexpr int SURCHARGE_PERCENT = 20;
constexpr int CONCESSION_PERCENT = 50;

bool isPeakHour(int hour) { return (hour >= 8 && hour < 10) || (hour >= 17 && hour <
20); }
bool isPeakTable(int hour) {                    // alternative: lookup table
   static constexpr bool PEAK[24] = {0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,1,1,1,0,0,0,0};
   return PEAK[hour];
}
long long withSurcharge(long long base, bool peak) { return peak ? base + base *
SURCHARGE_PERCENT / 100 : base; }
long long withConcession(long long fare, bool student, bool senior) {
   return (student || senior) ? fare - fare * CONCESSION_PERCENT / 100 : fare;
}

int main() {
   bool ok = true;
   for (int h = 0; h < 24; ++h) ok = ok && (isPeakHour(h) == isPeakTable(h));
   std::cout << "range-check and table agree for all 24 hours: " << std::boolalpha << ok <<
'\n';
   for (int h : {7, 8, 9, 10, 16, 17, 19, 20})
      std::cout << "hour " << h << " peak? " << isPeakHour(h) << '\n';
   long long f = withSurcharge(3000, true);
   std::cout << "Outer peak fare = " << f << " paise; student = " << withConcession(f, true,
false) << " paise\n";

   int i = 5;
   i = i++ + 1;                    // BAD: modifies i twice (undefined before C++17 rules;
unspecified result style)
   int j = 5; j++; j = j + 1;       // clear version
   std::cout << "bad style i=" << i << ", clear style j=" << j << '\n';
   int m = 7; m += 3 * 2;           // compound assignment: m = m + (3*2)
   std::cout << "m += 3*2 -> " << m << '\n';
}
```

Result: the range-check expression and the lookup table agree for all 24 hours (true). Hours 8, 9, 17 and 19 are peak; 7, 10, 16 and 20 are off-peak. Outer peak fare = 3000 + 600 = 3600 paise; the student concession halves it to 1800 paise. The warning “operation on ’i’ may be undefined [-Wsequence-point]” appears for the deliberately bad line i = i++ + 1; the clear style gives j = 7.

5.9 Alternative Concept and Complexity Analysis – lookup table bool peak[24]

Solution: static constexpr bool PEAK[24] = {0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,1,1,1,0,0,0,0}; and isPeakTable(hour) returns PEAK[hour]. The program verifies that it gives the same answer as the expression for all 24 hours.

Measure	&& / || expression	Lookup table peak[24]

Measure	&& / || expression	Lookup table peak[24]
Time per check	O(1): up to 4 comparisons and 3 logical operators (short-circuit)	O(1): one array index, one memory read
Space	None extra	24 bytes
Minute precision (e.g. 17:30)	Easy: use minutes since midnight	Needs 1440 entries (1440 bytes)
Changing the peak window	Edit two numbers	Edit several array entries; harder to read and check
Safety	No out-of-range risk	An hour outside 0–23 is an out-of-bounds read (undefined behaviour)

Task 6 – Design the Control Structures (4 Marks)

6.1 Discussion

- **if–else, switch on the zone and the conditional operator:** use if–else for ranges and compound conditions (the peak check, the Rs 50 rule). Use switch when one value is compared with a list of distinct constants, such as the three zones; with an enum class the compiler can warn (-Wswitch) when a zone is missing. Use the conditional operator ?: for a simple choice between two values (fare = peak ? f + f * 20 / 100 : f).
- **for, range-based for and while for processing journeys:** a classic for loop with an index is needed only when the position matters. A range-based for (for (const Trip& t : trips)) cannot go out of bounds, needs no index and does not copy (const reference), so it is **best** for a day's stored journeys. A while loop is right when the number of journeys is not known, e.g. while (std::cin >> zone >> hour) reads until the input ends.
- **break and continue:** continue skips an invalid journey and moves to the next one (if (!valid) continue;). break leaves a loop early (for example stopping the day's processing when the card is blocked or on an end-of-day code). In a switch, break prevents fall-through into the next case.
- **When no case of a switch matches:** nothing in the switch executes and control continues after it; with no default the variable keeps whatever value it had before. In fare() that would leave base at 0, i.e. a free journey. Task 12 demonstrates exactly this when the Airport zone is added: fare 0 and the warning -Wswitch. A default case that reports an error, or initialising to a harmless value and warning, is the safeguard.

6.2 Pseudocode for the fare computation

Pseudocode

```
ALGORITHM ComputeFare
INPUT:  zone (Inner, Middle or Outer), isPeak (true or false)
OUTPUT: fare in paise (integer)

  SET base ← 0
  SWITCH zone
    CASE Inner:   base ← BASE_PAISE[0]
    CASE Middle:  base ← BASE_PAISE[1]
    CASE Outer:   base ← BASE_PAISE[2]
  ENDSWITCH
  IF isPeak THEN
    base ← base + (base × PEAK_SURCHARGE_PERCENT) DIV 100
  ENDIF
  RETURN base
END ALGORITHM

ALGORITHM CanTravel
```

```
INPUT:  balance, fare
OUTPUT: true or false
  RETURN (balance − fare) ≥ MIN_BALANCE_PAISE
END ALGORITHM

ALGORITHM ProcessJourneys
  SET balance ← 10000
  FOR EACH trip IN trips
    SET f ← ComputeFare(trip.zone, trip.peak)
    IF CanTravel(balance, f) THEN
      balance ← balance − f
      PRINT "Fare Rs", f DIV 100, ".", two-digit (f MOD 100)
    ELSE
      PRINT "Denied (low balance)"
    ENDIF
    PRINT "balance Rs", balance DIV 100
  ENDFOR
END ALGORITHM
```

6.3 Complete the Code – TODO 1 and TODO 2 completed

File: Task06 / main.cpp

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;      // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
   long long base = 0;
   switch (z) {                              // TODO 1 completed
      case Zone::Inner:  base = BASE_PAISE[0]; break;
      case Zone::Middle: base = BASE_PAISE[1]; break;
      case Zone::Outer:  base = BASE_PAISE[2]; break;
      default: return -1; // reject an invalid zone value
   }
   if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
   return base;
}

bool canTravel(long long balance, long long f)
```

```
{
   return f >= 0 && balance >= MIN_BALANCE_PAISE && f <= balance -
MIN_BALANCE_PAISE;        // TODO 2 completed
}

int main()
{
   long long balance = 10000;                    // Rs 100.00 in paise
   struct Trip { Zone zone; bool peak; };
   Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
   for (const Trip& t : trips) {
      long long f = fare(t.zone, t.peak);
      if (canTravel(balance, f)) {
         balance -= f;
         std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                   << std::setfill('0') << f % 100 << std::setfill(' ');
      } else {
         std::cout << "Denied (low balance)";
      }
      std::cout << "  balance Rs " << balance / 100 << '\n';
   }
}
```

Terminal - Task 6 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra main.cpp -o main
$ ./main
Fare Rs 10.00  balance Rs 90
Fare Rs 36.00  balance Rs 54
Denied (low balance)  balance Rs 54
$ g++ -std=c++17 -Wall -Wextra boundary.cpp -o boundary
$ ./boundary
7:59 peak = false
8:00 peak = true
9:59 peak = true
10:00 peak = false
16:59 peak = false
17:00 peak = true
19:59 peak = true
20:00 peak = false
$ g++ -std=c++17 -Wall -Wextra array_version.cpp -o array_version
$ ./array_version
Airport peak = 7200 paise (only the table changed)
```

Screenshot 6: compile and run of Task06 (completed program, boundary test and array version)

Expected output given in the assignment:

```
Fare Rs 10.00  balance Rs 90
```

```
Fare Rs 36.00  balance Rs 54
Denied (low balance)  balance Rs 54
```

Comparison: the actual output of Task06/main.cpp is identical to the expected output, line for line. Working: Inner off-peak = 1000, balance 10000 − 1000 = 9000 (Rs 90). Outer peak = 3000 + 3000 × 20 / 100 = 3600, balance 9000 − 3600 = 5400 (Rs 54), and 5400 ≥ 5000. Middle off-peak = 2000: 5400 − 2000 = 3400 < 5000, so the journey is denied and the balance stays Rs 54.

6.4 Journeys at exactly 10 AM and 8 PM

File: Task06 / boundary.cpp

```
// Task 6 extension: journeys at exactly 10:00 AM and 8:00 PM (and 8:00, 17:00)
#include <iostream>

bool isPeak(int hour, int minute) {
   int t = hour * 60 + minute;                    // minutes since midnight
   return (t >= 8 * 60 && t < 10 * 60) || (t >= 17 * 60 && t < 20 * 60);
}
int main() {
   struct T { int h, m; } tests[] =
{{7,59},{8,0},{9,59},{10,0},{16,59},{17,0},{19,59},{20,0}};
   std::cout << std::boolalpha;
   for (auto t : tests)
      std::cout << t.h << ':' << (t.m < 10 ? "0" : "") << t.m << " peak = " << isPeak(t.h, t.m)
<< '\n';
}
```

Result (see Screenshot 6): 7:59 off-peak, 8:00 peak, 9:59 peak, 10:00 off-peak, 16:59 off-peak, 17:00 peak, 19:59 peak, 20:00 off-peak. Because the intervals are half-open, exactly 10:00 and exactly 20:00 are treated as off-peak consistently, and exactly 8:00 and 17:00 as peak, which removes the inconsistency in the first version of the software.

6.5 Alternative Concept and Complexity Analysis – fare looked up from an array

File: Task06 / array_version.cpp

```
// Alternative: fare looked up from an array indexed by zone, with a fourth zone added
#include <iostream>
enum class Zone { Inner, Middle, Outer, Airport };
constexpr long long FARE[] = {1000, 2000, 3000, 6000};
long long fare(Zone z, bool peak) {
   long long f = FARE[static_cast<int>(z)];
   return peak ? f + f * 20 / 100 : f;
}
int main() { std::cout << "Airport peak = " << fare(Zone::Airport, true) << " paise (only the
table changed)\n"; }
```

Measure	switch on the zone	array FARE[] indexed by zone
Time per lookup	O(1); the compiler may build a jump table (or a few compares)	O(1): one indexed read
Space	No data; code only	3 × 8 = 24 bytes for long long FARE[3]
Adding a fourth zone	Add an enum value and a new case (forgetting it leaves fare 0 and only a warning)	Add an enum value and one array element; nothing else changes (output shows Airport peak = 7200 with only the table edited)
Risk	Missing case	Index out of range if the table is shorter than the enum (undefined behaviour); guard with std::array and static_assert

Task 7 – From Procedural to Object-Oriented (4 Marks)

7.1 Procedural version (separate variables and free functions)

The balance is a global variable and deduct() simply subtracts. The caller is trusted to call canTravel() first.

File: Task07 / procedural.cpp

```
// Task 7(a): procedural version - global variable and free functions
#include <iostream>

long long balance = 10000;                    // global: any code can change it
const long long MIN_BALANCE = 5000;

long long fare(int zone, bool peak) {
   long long b = (zone == 0) ? 1000 : (zone == 1) ? 2000 : 3000;
   return peak ? b + b * 20 / 100 : b;
}
bool canTravel(long long f) { return balance - f >= MIN_BALANCE; }
void deduct(long long f) { balance -= f; }       // forgets to check canTravel!

int main() {
   deduct(fare(2, true));  std::cout << "after Outer peak : " << balance << '\n';
   deduct(fare(2, true));  std::cout << "after Outer peak : " << balance << '\n';
   deduct(fare(2, true));  std::cout << "after Outer peak : " << balance << " (below Rs 50
limit - rule broken)\n";
   balance = -999;         std::cout << "any code can set : " << balance << '\n';
}
```

7.2 Object-oriented version (SmartCard with private balance)

File: Task07 / oop.cpp

```
// Task 7(b): object-oriented version - SmartCard protects its balance
#include <iostream>

enum class Zone { Inner, Middle, Outer };

class SmartCard {
   long long balance_;
   static constexpr long long MIN_BALANCE = 5000;   // the only place the rule lives
public:
   explicit SmartCard(long long b) : balance_(b < 0 ? 0 : b) {}
   bool canTravel(long long f) const { return f >= 0 && balance_ >= MIN_BALANCE
&& f <= balance_ - MIN_BALANCE; }
```

```
    bool deduct(long long f) { if (!canTravel(f)) return false; balance_ -= f; return true; }
    long long balance() const { return balance_; }
};

long long fare(Zone z, bool peak) {
    long long b = 0;
    switch (z) { case Zone::Inner: b = 1000; break; case Zone::Middle: b = 2000; break; case
Zone::Outer: b = 3000; break; }
    return peak ? b + b * 20 / 100 : b;
}

int main() {
    SmartCard card(10000);
    for (int i = 1; i <= 3; ++i) {
        bool ok = card.deduct(fare(Zone::Outer, true));
        std::cout << "journey " << i << (ok ? " allowed" : " DENIED") << ", balance = " <<
card.balance() << '\n';
    }
    // card.balance_ = -5;   // compile-time ERROR: balance_ is private
}
```

Terminal - Task 7 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra procedural.cpp -o procedural
$ ./procedural
after Outer peak : 6400
after Outer peak : 2800
after Outer peak : -800 (below Rs 50 limit - rule broken)
any code can set : -999
$ g++ -std=c++17 -Wall -Wextra oop.cpp -o oop
$ ./oop
journey 1 allowed, balance = 6400
journey 2 DENIED, balance = 6400
journey 3 DENIED, balance = 6400
$ g++ -std=c++17 -Wall -Wextra public_member.cpp -o public_member
$ ./public_member
balance = -500 (invariants broken)
```

Screenshot 7: compile and run of Task07 (procedural, object-oriented and public-member programs)

7.3 How the class prevents a negative balance

- balance_ is private, so no code outside SmartCard can assign to it (the commented line card.balance_ = -5; is a compile-time error).
- deduct() calls canTravel() itself and returns false without changing anything if balance_ − fare would fall below MIN_BALANCE (Rs 50). Since Rs 50 > 0, the balance can never become negative through a journey.

- canTravel() also rejects a negative fare (f >= 0), and the constructor clamps a negative opening balance to 0.
- Evidence: in the object-oriented run the first Outer peak journey is allowed (balance 6400) and the second and third are DENIED with the balance still 6400. In the procedural run the same three calls leave 6400, 2800 and then −800: the balance went negative, and then −999 was assigned directly by unrelated code.

7.4 Which version is easier to change when the minimum balance changes

In the procedural version the number is stored once (MIN_BALANCE) but nothing forces its use: deduct() ignores it, so changing the constant changes nothing for callers that skip canTravel(), and each caller must be reviewed. In the object-oriented version the constant is a private static member used only inside canTravel(), and every deduction passes through it; changing it in one place changes the behaviour of the whole system and no caller needs to be edited. The object-oriented version is easier and safer to change.

7.5 Alternative Concept and Complexity Analysis – public balance member

File: Task07 / public_member.cpp

```
// Alternative: public balance - the rule can be broken by any part of the program
#include <iostream>
struct BadCard { long long balance; };
int main() {
   BadCard c{10000};
   c.balance -= 9990;                    // legal, but leaves Rs 0.10
   c.balance = -500;                     // legal, negative balance
   std::cout << "balance = " << c.balance << " (invariants broken)\n";
}
```

Solution: struct BadCard { long long balance; }; with free functions or direct access c.balance -= fare. Time and space do not change: reading or subtracting a public member costs the same O(1) as calling an inline member function (the compiler inlines deduct()), and the struct is still 8 bytes. What changes is correctness: the program printed balance = -500, showing that **any part of the program can break R1 and R2** (the minimum-balance rule and the no-negative-balance rule), and R8 (a denied journey leaves the balance unchanged) is unprotected. Encapsulation costs nothing at run time and removes this whole class of faults.

Task 8 – Read and Print Data Safely (4 Marks)

8.1 Discussion

- **Reading with cin and checking cin.fail():** if the operator types letters where a number is expected, the extraction cin >> v fails: v is not set, the stream goes into a fail state and all later reads do nothing until it is repaired. The expression if (std::cin >> v) tests the result of the read directly; std::cin.fail() tests the state afterwards. Note that cin >> v on “12abc” reads 12 successfully and leaves “abc” in the buffer, which makes the next read fail.
- **Clearing the error state and discarding the bad input:** std::cin.clear() resets the error flags, and std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n') throws away the rest of the bad line. Both are needed: clear() alone would read the same bad characters again forever.
- **Formatting money with <iomanip>:** std::setw(n) sets the field width for the next item, std::setfill('0') pads with zeros (so 5 paise prints as .05), std::fixed with std::setprecision(2) prints a double with exactly two decimals. Money is still held as integer paise and printed as paise / 100, a dot, and paise % 100 padded to two digits, so no double is involved in the stored value.
- **Printing a neat journey report:** a header row and one row per journey with left-aligned text columns (std::left) and right-aligned money columns (std::right with setw), followed by the closing balance.

8.2 Pseudocode for safe input

Pseudocode

```
ALGORITHM ReadInt
INPUT:  prompt, lowest, highest
OUTPUT: a valid integer in [lowest, highest]
  LOOP
    PRINT prompt
    READ value
    IF the read succeeded AND lowest ≤ value ≤ highest THEN
      RETURN value
    ELSE IF end of input THEN
      RETURN lowest
    ELSE IF the read failed THEN          // letters were typed
      CLEAR the error state
      DISCARD the rest of the line
      PRINT "invalid number, try again"
    ELSE
      PRINT "out of range, try again"
    ENDIF
  ENDLOOP
END ALGORITHM
```

8.3 Implementation

File: Task08 / main.cpp

```
// Task 8: safe input with cin.fail()/clear()/ignore() and formatted output
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>

bool echoInput = false;                         // --echo: print values read from a file/pipe
int readInt(const char* prompt, int lo, int hi) {
   int v;
   while (true) {
      std::cout << prompt;
      if (std::cin >> v && v >= lo && v <= hi) { if (echoInput) std::cout << v << '\n'; return
v; }
      if (std::cin.eof()) { std::cout << "\nend of input\n"; return lo; }
      if (std::cin.fail()) {                    // letters typed instead of numbers
         std::cin.clear();                      // reset the error state
         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // discard bad line
         std::cout << "  invalid number, try again\n";
      } else {
         std::cout << "  out of range (" << lo << ".." << hi << "), try again\n";
      }
   }
}
std::string money(long long paise) {            // "Rs 36.00" built with <iomanip>
   std::ostringstream o;
   o << "Rs " << paise / 100 << '.' << std::setw(2) << std::setfill('0') << paise % 100;
   return o.str();
}
int main(int argc, char**) {
   echoInput = argc > 1;
   const long long base[] = {1000, 2000, 3000};
   long long balance = 10000;
   std::cout << "=== Namma Metro Gate ===\n";
   int n = readInt("Number of journeys (1-3): ", 1, 3);
   int zs[3], hs[3];
   for (int i = 0; i < n; ++i) {
      zs[i] = readInt("Zone (0=Inner 1=Middle 2=Outer): ", 0, 2);
      hs[i] = readInt("Hour (0-23): ", 0, 23);
   }
   std::cout << '\n' << std::left << std::setw(4) << "No" << std::setw(8) << "Zone" <<
std::setw(7) << "Hour"
             << std::right << std::setw(12) << "Fare" << std::setw(14) << "Balance" << '\n';
   for (int i = 1; i <= n; ++i) {
      int zone = zs[i - 1], hour = hs[i - 1];
```

```
        bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
        long long f = base[zone]; if (peak) f += f * 20 / 100;
        std::cout << std::left << std::setw(4) << i << std::setw(8) << zone << std::setw(7) <<
hour << std::right;
        if (balance - f >= 5000) { balance -= f; std::cout << std::setw(12) << money(f) <<
std::setw(14) << money(balance); }
        else std::cout << std::setw(12) << "DENIED" << std::setw(14) << money(balance);
        std::cout << '\n';
    }
    std::cout << std::fixed << std::setprecision(2) << "Closing balance (double view): " <<
balance / 100.0 << '\n';
}
```

File: Task08 / getline_version.cpp

```
// Alternative: getline + stoi inside try/catch
#include <iostream>
#include <string>
int readInt(const char* prompt, int lo, int hi) {
    std::string line;
    while (true) {
        std::cout << prompt;
        if (!std::getline(std::cin, line)) return lo;
        try {
            size_t pos = 0;
            int v = std::stoi(line, &pos);
            if (pos != line.size()) throw std::invalid_argument("trailing");   // rejects "12abc"
            if (v < lo || v > hi) throw std::out_of_range("range");
            return v;
        } catch (const std::exception&) { std::cout << "  invalid, try again\n"; }
    }
}
int main() {
    int a = readInt("Hour (0-23): ", 0, 23);
    int b = readInt("Zone (0-2): ", 0, 2);
    std::cout << "hour=" << a << " zone=" << b << '\n';
}
```

8.4 Input files and execution

input_valid.txt contains 3 journeys (zones 0, 2, 1 at hours 7, 18, 12). input_invalid.txt contains bad entries: abc (letters for the count), zone (letters for a zone), 99 and 25 (out-of-range hour). The program is run with --echo so that values taken from the input file are shown on screen as if typed.

Terminal - Task 8 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra main.cpp -o main
$ ./main --echo < input_valid.txt
=== Namma Metro Gate ===
Number of journeys (1-3): 3
Zone (0=Inner 1=Middle 2=Outer): 0
Hour (0-23): 7
Zone (0=Inner 1=Middle 2=Outer): 2
Hour (0-23): 18
Zone (0=Inner 1=Middle 2=Outer): 1
Hour (0-23): 12

No  Zone    Hour           Fare       Balance
1   0       7          Rs 10.00      Rs 90.00
2   2       18         Rs 36.00      Rs 54.00
3   1       12           DENIED      Rs 54.00
Closing balance (double view): 54.00
$ ./main --echo < input_invalid.txt
=== Namma Metro Gate ===
Number of journeys (1-3):   invalid number, try again
Number of journeys (1-3): 3
Zone (0=Inner 1=Middle 2=Outer):   invalid number, try again
Zone (0=Inner 1=Middle 2=Outer): 1
Hour (0-23):   out of range (0..23), try again
Hour (0-23):   out of range (0..23), try again
Hour (0-23): 9
Zone (0=Inner 1=Middle 2=Outer): 0
Hour (0-23): 12
Zone (0=Inner 1=Middle 2=Outer): 2
Hour (0-23): 18

No  Zone    Hour           Fare       Balance
1   1       9          Rs 24.00      Rs 76.00
2   0       12         Rs 10.00      Rs 66.00
3   2       18           DENIED      Rs 66.00
Closing balance (double view): 66.00
$ g++ -std=c++17 -Wall -Wextra getline_version.cpp -o getline_version
$ ./getline_version < input_getline.txt
Hour (0-23):   invalid, try again
Hour (0-23):   invalid, try again
Hour (0-23):   invalid, try again
Hour (0-23): Zone (0-2):   invalid, try again
Zone (0-2): hour=10 zone=1
```

Screenshot 8: compile and run of Task08 with valid input, invalid input and the getline/stoi version

Result: with the valid file the report shows Rs 10.00 → Rs 90.00, Rs 36.00 → Rs 54.00 and the third journey DENIED (balance would fall below Rs 50). With the invalid file the program printed “invalid number, try again” for abc and zone and “out of range” for 99 and 25, recovered every time, and finally produced the correct report (Rs 24.00 for Middle at 09:00 peak, Rs 10.00 for Inner at 12:00, and Outer at 18:00 denied since 6600 − 3600 = 3000 < 5000). The program never crashed or looped forever.

8.5 Alternative Concept and Complexity Analysis – getline and std::stoi with try/catch

Solution: read the whole line with std::getline, convert with std::stoi inside try/catch, reject trailing characters by checking the position returned by stoi and reject out-of-range values (Task08/getline_version.cpp). Input tried: ten, 12abc, 99, 10 (hour) and 7, 1 (zone): the program rejected ten, 12abc and 99, accepted 10; rejected 7, accepted 1.

Measure	**cin >> int with fail()/clear()/ignore()**	**getline + stoi + try/catch**
Time for a line of length L	O(L): characters are scanned once	O(L): the line is copied into a string, then scanned once by stoi
Buffer space	O(1) extra (stream buffer only)	O(L): the std::string holding the line
Recovery from invalid input	clear() + ignore() to end of line	Nothing to clear; the stream is never put in a failed state; the next getline reads the next line
Input such as 12abc	Accepts 12 and leaves abc to break the next read	Rejected completely by the pos != line.size() check
Cost of an invalid line	Cheap	An exception is thrown: slower, but only for bad input

Task 9 – Implement and Test the Fare Logic (4 Marks)

9.1 Design and pseudocode

Pseudocode

```
ALGORITHM Fare
INPUT:  zone, hour, minute, hasConcession
OUTPUT: fare in paise
  SET t ← hour × 60 + minute
  SET peak ← (t ≥ 480 AND t < 600) OR (t ≥ 1020 AND t < 1200)
  SET f ← BASE_PAISE[zone]
  IF peak THEN f ← f + f × 20 DIV 100 ENDIF
  IF hasConcession THEN f ← f − f × 50 DIV 100 ENDIF
  RETURN f
END ALGORITHM

ALGORITHM Deduct
INPUT:  balance, fare
  IF balance − fare ≥ 5000 THEN balance ← balance − fare; RETURN true
  ELSE RETURN false (balance unchanged)
  ENDIF
END ALGORITHM
```

9.2 Implementation (complete fare logic with validation and tests)

File: Task09 / main.cpp

```
// Task 9: complete fare logic with validation and tests
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

enum class Zone { Inner = 0, Middle = 1, Outer = 2 };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int  PEAK_SURCHARGE_PERCENT = 20;
constexpr int  CONCESSION_PERCENT = 50;
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeak(int hour, int minute) {
    int t = hour * 60 + minute;
    return (t >= 8 * 60 && t < 10 * 60) || (t >= 17 * 60 && t < 20 * 60);
```

```
}
bool validZone(int z) { return z >= 0 && z <= 2; }

long long fare(Zone z, bool peak, bool concession) {
    const int index = static_cast<int>(z);
    if (index < 0 || index >= 3) return -1;
    long long f = BASE_PAISE[index];
    if (peak) f += f * PEAK_SURCHARGE_PERCENT / 100;
    if (concession) f -= f * CONCESSION_PERCENT / 100;
    return f;
}

class SmartCard {
    long long balance_;
public:
    explicit SmartCard(long long b) : balance_(b < 0 ? 0 : b) {}
    bool canTravel(long long f) const { return f >= 0 && balance_ >=
MIN_BALANCE_PAISE && f <= balance_ - MIN_BALANCE_PAISE; }
    bool deduct(long long f) { if (!canTravel(f)) return false; balance_ -= f; return true; }
    bool recharge(long long amt) { if (amt <= 0) return false; balance_ += amt; return true; }
    long long balance() const { return balance_; }
};

std::string money(long long p) {
    std::ostringstream o;
    o << "Rs " << p / 100 << '.' << std::setw(2) << std::setfill('0') << p % 100;
    return o.str();
}

// parse "zone hour" text safely; returns false on bad input
bool parseJourney(const std::string& line, int& zone, int& hour) {
    std::istringstream in(line);
    if (!(in >> zone >> hour)) return false;
    in >> std::ws;
    return in.eof() && validZone(zone) && hour >= 0 && hour <= 23;
}

int main() {
    int pass = 0, fail = 0;
    auto check = [&](const std::string& name, long long expected, long long actual) {
        bool ok = expected == actual;
        (ok ? pass : fail)++;
        std::cout << std::left << std::setw(46) << name << std::right << std::setw(7) << expected
```

```
              << std::setw(8) << actual << "   " << (ok ? "PASS" : "FAIL") << '\n';
    };
    std::cout << std::left << std::setw(46) << "Test case" << std::right << std::setw(7) <<
"Expect"
              << std::setw(8) << "Actual" << "   Result\n" << std::string(72, '-') << '\n';

    // A. normal journeys in each zone, off-peak and peak (paise)
    check("Inner off-peak 12:00", 1000, fare(Zone::Inner, isPeak(12,0), false));
    check("Inner peak 08:30", 1200, fare(Zone::Inner, isPeak(8,30), false));
    check("Middle off-peak 14:00", 2000, fare(Zone::Middle, isPeak(14,0), false));
    check("Middle peak 18:00", 2400, fare(Zone::Middle, isPeak(18,0), false));
    check("Outer off-peak 21:00", 3000, fare(Zone::Outer, isPeak(21,0), false));
    check("Outer peak 17:30", 3600, fare(Zone::Outer, isPeak(17,30), false));
    check("Student Outer peak 17:30", 1800, fare(Zone::Outer, true, true));
    // B. boundaries
    check("Boundary 08:00 is peak (Inner)", 1200, fare(Zone::Inner, isPeak(8,0), false));
    check("Boundary 10:00 is off-peak (Inner)", 1000, fare(Zone::Inner, isPeak(10,0), false));
    check("Boundary 17:00 is peak (Inner)", 1200, fare(Zone::Inner, isPeak(17,0), false));
    check("Boundary 20:00 is off-peak (Inner)", 1000, fare(Zone::Inner, isPeak(20,0), false));
    check("Boundary 07:59 off-peak", 0, isPeak(7,59));
    check("Boundary 19:59 peak", 1, isPeak(19,59));
    {   SmartCard c(6000);   // Rs 60, fare Rs 10 -> leaves exactly Rs 50
        check("Leaves exactly Rs 50 -> allowed", 1, c.canTravel(1000));
        SmartCard d(5999);
        check("Leaves Rs 49.99 -> denied", 0, d.canTravel(1000)); }
    // C. invalid input
    int z = 0, h = 0;
    check("Unknown zone '5 9' rejected", 0, parseJourney("5 9", z, h));
    check("Letters for hour '1 ab' rejected", 0, parseJourney("1 ab", z, h));
    check("Hour 24 rejected", 0, parseJourney("1 24", z, h));
    check("Valid '2 18' accepted", 1, parseJourney("2 18", z, h));
    { SmartCard c(10000); check("Negative recharge rejected", 0, c.recharge(-500));
      check("Balance unchanged after bad recharge", 10000, c.balance()); }
    // D. five journeys calculated by hand: total = 1000+2400+3000+3600+1200 = 11200
    SmartCard card(50000);
    struct J { Zone z; int h, m; } js[] =
{{Zone::Inner,10,0},{Zone::Middle,8,30},{Zone::Outer,12,0},{Zone::Outer,17,0},{Zone::In
ner,19,59}};
    long long total = 0;
    for (const J& j : js) { long long f = fare(j.z, isPeak(j.h, j.m), false); if (card.deduct(f)) total
+= f; }
    check("5 journeys hand total (Rs 112.00)", 11200, total);
    check("Balance after 5 journeys (Rs 388.00)", 38800, card.balance());
```

```
    std::cout << std::string(72, '-') << "\nPassed " << pass << ", failed " << fail << "\n";
    std::cout << "Example money format: " << money(11200) << "  " << money(1205) << '\n';
    return fail == 0 ? 0 : 1;
}
```

File: Task09 / table_tests.cpp

```
// Alternative: table-driven tests - a new boundary case is one new row
#include <iostream>
#include <vector>
bool isPeak(int h, int m) { int t = h*60+m; return (t>=480&&t<600)||(t>=1020&&t<1200); }
struct Case { int h, m; bool expected; };
int main() {
    std::vector<Case> cases =
{{7,59,false},{8,0,true},{9,59,true},{10,0,false},{16,59,false},{17,0,true},{19,59,true},{20,0
,false},{0,0,false},{23,59,false}};
    int bad = 0;
    for (const Case& c : cases) if (isPeak(c.h, c.m) != c.expected) { ++bad; std::cout << "FAIL
" << c.h << ':' << c.m << '\n'; }
    std::cout << cases.size() << " cases, " << bad << " failures\n";
}
```

9.3 Execution

Terminal - Task 9 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra main.cpp -o main
$ ./main
Test case                                        Expect  Actual   Result
------------------------------------------------------------------------
Inner off-peak 12:00                               1000    1000   PASS
Inner peak 08:30                                   1200    1200   PASS
Middle off-peak 14:00                              2000    2000   PASS
Middle peak 18:00                                  2400    2400   PASS
Outer off-peak 21:00                               3000    3000   PASS
Outer peak 17:30                                   3600    3600   PASS
Student Outer peak 17:30                           1800    1800   PASS
Boundary 08:00 is peak (Inner)                     1200    1200   PASS
Boundary 10:00 is off-peak (Inner)                 1000    1000   PASS
Boundary 17:00 is peak (Inner)                     1200    1200   PASS
Boundary 20:00 is off-peak (Inner)                 1000    1000   PASS
Boundary 07:59 off-peak                               0       0   PASS
Boundary 19:59 peak                                   1       1   PASS
Leaves exactly Rs 50 -> allowed                       1       1   PASS
Leaves Rs 49.99 -> denied                             0       0   PASS
Unknown zone '5 9' rejected                           0       0   PASS
Letters for hour '1 ab' rejected                      0       0   PASS
Hour 24 rejected                                      0       0   PASS
Valid '2 18' accepted                                 1       1   PASS
Negative recharge rejected                            0       0   PASS
Balance unchanged after bad recharge              10000   10000   PASS
5 journeys hand total (Rs 112.00)                 11200   11200   PASS
Balance after 5 journeys (Rs 388.00)              38800   38800   PASS
------------------------------------------------------------------------
Passed 23, failed 0
Example money format: Rs 112.00  Rs 12.05
$ g++ -std=c++17 -Wall -Wextra table_tests.cpp -o table_tests
$ ./table_tests
10 cases, 0 failures
```

Screenshot 9: compile and run of Task09 (23 tests, all PASS) and the table-driven alternative

9.4 Test table with expected and actual results

No.	Test case	Expected	Actual	Result
1	Inner off-peak 12:00	1000	1000	PASS
2	Inner peak 08:30	1200	1200	PASS
3	Middle off-peak 14:00	2000	2000	PASS
4	Middle peak 18:00	2400	2400	PASS
5	Outer off-peak 21:00	3000	3000	PASS
6	Outer peak 17:30	3600	3600	PASS
7	Student Outer peak 17:30	1800	1800	PASS
8	Boundary 08:00 is peak (Inner)	1200	1200	PASS
9	Boundary 10:00 is off-peak (Inner)	1000	1000	PASS
10	Boundary 17:00 is peak (Inner)	1200	1200	PASS
11	Boundary 20:00 is off-peak (Inner)	1000	1000	PASS

No.	Test case	Expected	Actual	Result
12	Boundary 07:59 off-peak	0	0	PASS
13	Boundary 19:59 peak	1	1	PASS
14	Leaves exactly Rs 50 -> allowed	1	1	PASS
15	Leaves Rs 49.99 -> denied	0	0	PASS
16	Unknown zone '5 9' rejected	0	0	PASS
17	Letters for hour '1 ab' rejected	0	0	PASS
18	Hour 24 rejected	0	0	PASS
19	Valid '2 18' accepted	1	1	PASS
20	Negative recharge rejected	0	0	PASS
21	Balance unchanged after bad recharge	10000	10000	PASS
22	5 journeys hand total (Rs 112.00)	11200	11200	PASS
23	Balance after 5 journeys (Rs 388.00)	38800	38800	PASS

Values are in paise unless the case is a yes/no check, where 1 = true and 0 = false. The tests cover: normal journeys in each zone, off-peak and peak (cases 1–7); boundaries 08:00, 10:00, 17:00, 20:00, 07:59, 19:59 and a balance leaving exactly Rs 50 or Rs 49.99 (8–15); invalid cases: unknown zone, letters typed for the hour, hour 24, a negative recharge (16–21); and the five-journey hand calculation (22–23).

9.5 Five journeys calculated by hand

No.	Journey	Calculation	Fare (paise)	Balance (Rs)
–	Opening balance	Rs 500.00	–	500.00
1	Inner, 10:00 (off-peak)	base 1000	1000	490.00
2	Middle, 08:30 (peak)	2000 + 20% = 2400	2400	466.00
3	Outer, 12:00 (off-peak)	base 3000	3000	436.00
4	Outer, 17:00 (peak)	3000 + 20% = 3600	3600	400.00
5	Inner, 19:59 (peak)	1000 + 20% = 1200	1200	388.00
	Total	1000 + 2400 + 3000 + 3600 + 1200	11200 (Rs 112.00)	388.00

The program’s results (11200 paise total and 38800 paise = Rs 388.00 balance) match the hand calculation exactly. Result of Task 9: 23 of 23 tests pass.

9.6 Alternative Concept and Complexity Analysis – table-driven tests

Solution: a vector of structs {hour, minute, expected} and one loop that runs every case (Task09/table_tests.cpp, 10 boundary cases, 0 failures).

Measure	Separate hand-written checks	Table of test cases in a loop
Time for t tests	O(t): one call per check	O(t): one loop iteration per case

Measure	Separate hand-written checks	Table of test cases in a loop
Space	None for data (code grows with t)	O(t): about 12 bytes per case (two ints and a bool); 120 bytes for 10 cases
Adding a new boundary case	Write a new statement and a new message	Add one row of data
Risk of copy-paste mistakes	Higher	Lower: the checking code is written once

Task 10 – Compare Expectations with Execution (4 Marks)

10.1 What I expected

- Integer-paise and double-rupee totals should agree for whole-rupee fares, but I expected double to show small differences when fractions such as 1.2 × Rs are used.
- int would be enough for 20,000 and probably 500,000 journeys but would overflow at 3,000,000 (the average fare is about Rs 21, so about 6.3 billion paise > 2.147 billion).
- The final code should compile without warnings under -Wall -Wextra.

10.2 Implementation and execution

File: Task10 / main.cpp

```
// Task 10: expected vs actual totals - double rupees vs integer paise, int overflow, timing
#include <chrono>
#include <climits>
#include <cstdint>
#include <iomanip>
#include <iostream>

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr double BASE_RUPEES[] = {10.0, 20.0, 30.0};

int main() {
   std::cout << std::fixed;
   std::cout << "Journey mix: zone = i%3, peak when i%4==0 (20% surcharge)\n\n";
   std::cout << std::left << std::setw(12) << "Journeys" << std::setw(20) << "paise total"
<< std::setw(24)
            << "double Rs total" << std::setw(18) << "difference (Rs)" << "fits int?\n";
   for (long long n : {20000LL, 500000LL, 3000000LL}) {
      long long p = 0; double r = 0.0;
      for (long long i = 0; i < n; ++i) {
         int z = static_cast<int>(i % 3); bool peak = (i % 4 == 0);
         long long f = BASE_PAISE[z]; if (peak) f += f * 20 / 100;
         double g = BASE_RUPEES[z]; if (peak) g *= 1.2;          // 1.2 is not exact in binary
         p += f; r += g;
      }
      double diff = r - p / 100.0;
      std::cout << std::left << std::setw(12) << n << std::setw(20) << p <<
std::setprecision(6) << std::setw(24) << r
               << std::scientific << std::setprecision(3) << std::setw(18) << diff << std::fixed
<< (p <= INT_MAX ? "yes" : "NO (overflow)") << '\n';
   }

   std::cout << "\nMix B (Task 12 style): even journeys get 10% weekend discount and
50% concession, odd pay full\n";
```

```
   std::cout << std::left << std::setw(12) << "Journeys" << std::setw(20) << "paise total"
<< std::setw(24) << "double Rs total" << "difference (Rs)\n";
   for (long long n : {20000LL, 500000LL, 3000000LL}) {
      long long p = 0; double r = 0.0;
      for (long long i = 0; i < n; ++i) {
         int z = static_cast<int>(i % 3);
         long long f = BASE_PAISE[z]; double g = BASE_RUPEES[z];
         if (i % 2 == 0) { f = f * 90 / 100; f = f * 50 / 100; g = g * 0.9; g = g * 0.5; }
         p += f; r += g;
      }
      std::cout << std::left << std::setw(12) << n << std::setw(20) << p <<
std::setprecision(6) << std::setw(24) << r
                << std::scientific << std::setprecision(3) << r - p / 100.0 << std::fixed << '\n';
   }

   // What an int accumulator would hold at 3,000,000 journeys (wraps modulo 2^32)
   std::uint32_t wrap = 0; long long exact = 0; float fl = 0.0f; double db = 0.0;
   for (long long i = 0; i < 3000000; ++i) {
      long long f = BASE_PAISE[i % 3]; if (i % 4 == 0) f += f * 20 / 100;
      wrap += static_cast<std::uint32_t>(f); exact += f; fl += static_cast<float>(f); db +=
static_cast<double>(f);
   }
   std::cout << "\nNetwork total, 3,000,000 journeys (paise):\n";
   std::cout << "  long long : " << exact << "  (correct)\n";
   std::cout << "  int (wrap): " << static_cast<std::int32_t>(wrap) << "  (OVERFLOW:
wrong value)\n";
   std::cout << "  double    : " << std::setprecision(1) << db << "  (exact here: integers <
2^53)\n";
   std::cout << "  float     : " << fl << "  (loses precision)\n";

   // time per addition
   using clk = std::chrono::steady_clock;
   const long long N = 200000000LL;
   volatile std::int32_t vi = 0; volatile long long vl = 0; volatile double vd = 0;
   auto t0 = clk::now(); for (long long i = 0; i < N; ++i) vi = vi + 1;
   auto t1 = clk::now(); for (long long i = 0; i < N; ++i) vl = vl + 1;
   auto t2 = clk::now(); for (long long i = 0; i < N; ++i) vd = vd + 1.0;
   auto t3 = clk::now();
   auto ns = [&](auto a, auto b) { return std::chrono::duration<double, std::nano>(b -
a).count() / N; };
   std::cout << "\nTime per addition over " << N << " additions (volatile, -O0 style):\n" <<
std::setprecision(3)
             << "  int       " << ns(t0, t1) << " ns   (4 bytes)\n"
             << "  long long " << ns(t1, t2) << " ns   (8 bytes)\n"
             << "  double    " << ns(t2, t3) << " ns   (8 bytes)\n";
}
```

Terminal - Task 10 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra main.cpp -o main
$ ./main
Journey mix: zone = i%3, peak when i%4==0 (20% surcharge)

Journeys    paise total         double Rs total         difference (Rs)   fits int?
20000       41998800            419988.000000           0.000e+00         yes
500000      1049998800          10499988.000000         0.000e+00         yes
3000000     6300000000          63000000.000000         0.000e+00         NO (overflow)

Mix B (Task 12 style): even journeys get 10% weekend discount and 50% concession, odd pay full
Journeys    paise total         double Rs total         difference (Rs)
20000       28999550            289995.500000           0.000e+00
500000      724999550           7249995.500000          0.000e+00
3000000     4350000000          43500000.000000         0.000e+00

Network total, 3,000,000 journeys (paise):
  long long : 6300000000  (correct)
  int (wrap): 2005032704  (OVERFLOW: wrong value)
  double    : 6300000000.0  (exact here: integers < 2^53)
  float     : 6367037952.0  (loses precision)

Time per addition over 200000000 additions (volatile, -O0 style):
  int       0.527 ns   (4 bytes)
  long long 0.525 ns   (8 bytes)
  double    2.189 ns   (8 bytes)
```

Screenshot 10: compile and run of Task10/main.cpp (totals, overflow demonstration and timing)

10.3 Results

Journeys	Integer paise total	Double rupee total (Rs)	Difference	Fits in int?
20,000	41,998,800 (Rs 419,988.00)	419,988.00	0	Yes
500,000	1,049,998,800 (Rs 10,499,988.00)	10,499,988.00	0	Yes (just under 2,147,483,647)
3,000,000	6,300,000,000 (Rs 63,000,000.00)	63,000,000.00	0	No: overflow

Mix used: journey i uses zone i mod 3 and is peak when i mod 4 = 0. A second mix with 10% weekend discount and 50% concession also gave zero difference (Screenshot 10).

10.4 Discussion points

- **Do the double and integer-paise totals agree, and by how much do they differ?** Yes: the difference is exactly 0 for 20,000, 500,000 and 3,000,000 journeys. This is not a proof that double is safe: every amount here (10, 12, 20, 24, 30, 36, 4.5, 13.5 …) is exactly representable in binary and the totals are far below 2^53, so no rounding occurs. Task 3 shows that the very same double type gives an error of 1.3 × 10^-6 when 0.10 is added a million times.

- **Does int overflow for the network total?** Yes. The 3,000,000-journey total is 6,300,000,000 paise, larger than INT_MAX. A 32-bit accumulator wraps to 2,005,032,704, a plausible-looking but wrong number, with no error message. Signed overflow is formally undefined behaviour, so the demonstration used an unsigned 32-bit wrap.
- **Compiler warnings received with -Wall -Wextra and what each meant:** (1) -Wsign-compare in Task 4: an int was compared with an unsigned int; the int is converted to unsigned, so -1 looks huge. (2) -Wsequence-point in Task 5: i = i++ + 1 modifies and uses i with no defined ordering. (3) -Wswitch in Task 12's impact demo: the enumerator Airport has no case in the switch. (4) -Wunused-parameter appeared once in Task 8 (argv declared but unused) and was fixed by leaving the parameter unnamed. The final programs of all tasks compile with no warnings (the intentional ones above are demonstrations). The three -Wnarrowing messages in Task 4 are errors from deliberately wrong code.
- **Reasons for any difference:** the only differences in this task are between the 32-bit wrapped value and the true total (a range problem) and between float and the true total (a precision problem: 6,367,037,952 against 6,300,000,000, i.e. 67 million paise too much, because float has only 24 bits of mantissa).

10.5 Alternative Concept and Complexity Analysis – network total in int, long long or double

Accumulator	Result at 3,000,000 journeys	When it fails	Time per addition (measured)	Space
int (32-bit)	2,005,032,704 (wrong)	Overflow when the total exceeds 2,147,483,647 paise (about Rs 21.4 million, already after about 1 million journeys)	0.53 ns	4 bytes
long long	6,300,000,000 (correct)	Overflow only above 9.22 × 10^18	0.53 ns	8 bytes
double	6,300,000,000.0 (correct here)	Loss of precision: above 2^53 ≈ 9.0 × 10^15 integer paise cannot all be represented; fractions fail much earlier	2.19 ns	8 bytes
float	6,367,037,952 (wrong)	Loses precision above 2^24 ≈ 16.7 million	about the same as double	4 bytes

Overflow is a range failure (the value does not fit); loss of precision is a representation failure (the value fits but not exactly). Per addition, all are O(1); the integer types are faster and long long is the only one that is both exact and large enough.

Task 11 – Recommend and Defend the Final Design (4 Marks)

11.1 The final design: types, constants and classes with reasons

Element	Recommendation	Evidence / reason
Money type	long long counting paise	Task 3: double gave 1.3e-06 error after 10^6 additions of Rs 0.10; Task 10: int overflowed at 3,000,000 journeys
Fare constants	constexpr std::array in namespace Rules (BASE_PAISE, PEAK_SURCHARGE_PERCENT, CONCESSION_PERCENT, MIN_BALANCE_PAISE)	Task 2: typed, scoped, checked by compiler, defined in one place
Zones	enum class Zone	Task 2: no accidental conversion to int; Task 12: adding Airport is an enum value plus one table entry
SmartCard class	private balance_, public canTravel(), deduct()	Task 7: the procedural version went to −800; the class kept the balance at 6400
Peak rule	Minutes since midnight with half-open intervals	Tasks 6 and 9: 10:00 and 20:00 off-peak, 08:00 and 17:00 peak, tested
Control structures	Range-based for over journeys, switch/array for zones, if for rules	Task 6
Input	cin with fail()/clear()/ignore() (or getline + stoi)	Task 8: recovered from bad input
Conversions	Brace initialisation and static_cast	Task 4: narrowing is a compile error

11.2 How a fare revision will be carried out

- Edit the one constant in namespace Rules (for example BASE_PAISE or PEAK_SURCHARGE_PERCENT); no other source line contains a fare value.
- Recompile with g++ -std=c++17 -Wall -Wextra and make sure there are no warnings.
- Run the test program (23 tests in Task 9, 14 in Task 12); update the expected values in the test table, which is also a record of the revision.
- Commit to the repository with a message that states the old and the new fare, then deploy the new build to the gates.

11.3 Limitations of the design

- Fares are compiled in: every revision needs a rebuild and redeployment (see the configuration-file analysis below).

- Percentages use integer division, which truncates. All current fares are multiples of 100 paise so the results are exact, but a future fare such as 1,050 paise at 15% would need an explicit rounding rule.
- The concession percentage (50%) is an assumption because the scenario did not give it.
- The program is single-threaded; real gates run in parallel and the shared balance would need locking or a central server.
- It does not persist card data, check card authenticity or handle the exit gate, tap-in/tap-out pairing or fare capping.

11.4 Is the design still correct for the whole network?

Yes, with long long. The worst-case network day is 3,000,000 × 3600 = 10.8 × 10^9 paise, which is far below the long long maximum (9.22 × 10^18); the measured total for the test mix was 6,300,000,000 paise and was exact. The time per journey is O(1) (about 6.5 ns for the complete Task 12 fare calculation including the loop, i.e. about 20 ms for 3,000,000 journeys), and the memory per card is 8 bytes (measured), so the design scales linearly with the number of journeys. A plain int would not be correct for the network total.

11.5 Implementation of the final design and configuration-file comparison

File: Task11 / main.cpp

```
// Task 11: recommended final design + comparison with a configuration-file fare table
#include <array>
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

enum class Zone { Inner = 0, Middle = 1, Outer = 2 };

namespace Rules {                                   // every rule value lives here only
   constexpr std::array<long long, 3> BASE_PAISE{1000, 2000, 3000};
   constexpr int  PEAK_SURCHARGE_PERCENT = 20;
   constexpr int  CONCESSION_PERCENT = 50;
   constexpr long long MIN_BALANCE_PAISE = 5000;
}
inline bool isPeak(int h, int m) { int t = h * 60 + m; return (t >= 480 && t < 600) || (t >=
1020 && t < 1200); }

long long fare(Zone z, bool peak, bool concession) {
   const auto index = static_cast<std::size_t>(z);
   if (index >= Rules::BASE_PAISE.size()) return -1;
```

```
    long long f = Rules::BASE_PAISE[index];
    if (peak) f += f * Rules::PEAK_SURCHARGE_PERCENT / 100;
    if (concession) f -= f * Rules::CONCESSION_PERCENT / 100;
    return f;
}

class SmartCard {
    long long balance_;
public:
    explicit SmartCard(long long b) : balance_(b) {}
    bool canTravel(long long f) const { return f >= 0 && balance_ - f >=
Rules::MIN_BALANCE_PAISE; }
    bool deduct(long long f) { if (!canTravel(f)) return false; balance_ -= f; return true; }
    long long balance() const { return balance_; }
};

// loads z fares from a text file; returns how many were read
std::size_t loadFares(const std::string& path, std::vector<long long>& out) {
    std::ifstream in(path);
    if (!in) return 0;
    std::string line;
    while (std::getline(in, line)) {
        if (line.empty() || line[0] == '#') continue;
        try {
            std::size_t used = 0;
            long long value = std::stoll(line, &used);
            if (used != line.size() || value < 0) return 0;
            out.push_back(value);
        } catch (const std::exception&) { return 0; }
    }
    return out.size();
}

int main() {
    // 1. final design in action
    SmartCard card(10000);
    long long f = fare(Zone::Outer, isPeak(18, 0), false);
    std::cout << "Outer, 18:00 fare = " << f << " paise, allowed = " << std::boolalpha <<
card.deduct(f)
              << ", balance = " << card.balance() << '\n';

    // 2. configuration-file alternative: read the three fares at start-up
    std::vector<long long> cfg;
```

```
    const std::size_t loaded = loadFares("fares.cfg", cfg);
    std::cout << "Fares read from fares.cfg: " << loaded;
    if (loaded >= 3) std::cout << " -> " << cfg[0] << ", " << cfg[1] << ", " << cfg[2];
    else std::cout << " (configuration file missing or incomplete)";
    std::cout << '\n';

    // 3. start-up time for z = 3 and z = 100000 zone values (generated file)
    for (std::size_t z : {std::size_t(3), std::size_t(1000), std::size_t(100000)}) {
        { std::ofstream o("big.cfg"); for (std::size_t i = 0; i < z; ++i) o << (1000 + 10 * i) <<
'\n'; }
        std::vector<long long> v; v.reserve(z);
        auto t0 = std::chrono::steady_clock::now();
        loadFares("big.cfg", v);
        auto t1 = std::chrono::steady_clock::now();
        std::cout << "z = " << std::setw(6) << z << "  load time = " << std::fixed <<
std::setprecision(1)
                  << std::chrono::duration<double, std::micro>(t1 - t0).count() << " us, memory =
"
                  << v.size() * sizeof(long long) << " bytes\n";
    }
    std::remove("big.cfg");
    std::cout << "Compiled-in table: 0 us start-up, " << sizeof(Rules::BASE_PAISE) << "
bytes, but needs recompile to revise\n";
}
```

File: Task11 / fares.cfg

```
# zone base fares in paise (one per line)
1000
2000
3000
```

Terminal - Task 11 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra main.cpp -o main
$ ./main
Outer, 18:00 fare = 3600 paise, allowed = true, balance = 6400
Fares read from fares.cfg: 3 -> 1000, 2000, 3000
z =      3  load time = 2.6 us, memory = 24 bytes
z =   1000  load time = 107.0 us, memory = 8000 bytes
z = 100000  load time = 5504.3 us, memory = 800000 bytes
Compiled-in table: 0 us start-up, 24 bytes, but needs recompile to revise
```

Screenshot 11: compile and run of Task11/main.cpp (final design and configuration-file start-up measurements)

11.6 Alternative Concept and Complexity Analysis – fare table read from a configuration file

Solution: at start-up open fares.cfg, read z lines of paise values into a std::vector<long long>, validate each (positive, within range), then use the vector exactly as the compiled table. A fare revision becomes editing a text file and restarting the gate.

Measure	Compiled-in constexpr table	Read from fares.cfg at start-up
Start-up time to read z fares	None (O(1), zero at run time)	O(z): measured 2.6 µs for z = 3; 107 µs for z = 1,000; 5,504 µs (5.5 ms) for z = 100,000
Space for z fares	8z bytes in read-only data (24 bytes for z = 3)	8z bytes on the heap plus vector overhead: 24 B, 8,000 B, 800,000 B for the same z
Cost of a fare revision	Edit, recompile, test and redeploy the program to every gate	Edit one file and restart; no recompile
Risk	None at run time	Missing, corrupt or wrongly edited file (need validation); fare values no longer protected by the compiler

Recommendation: for three zones the compiled-in table is better: zero start-up cost, type-checked, and the five-minute rebuild is negligible. If the metro grows to many zones or revises fares often, a validated configuration file becomes worthwhile, because the 5.5 ms start-up cost for 100,000 values is trivial compared with redeploying software to every gate.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

Changed requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones. Decision: **MODIFY** the original design (retain the classes, paise and constants; modify the fare logic and add a Pass class). Reasons are given in parts B and D.

A. Impact Analysis (2 Marks)

Part of the design	Affected?	What must change
Constants	Yes	BASE_PAISE gets a fourth entry 6000; new WEEKEND_DISCOUNT_PERCENT = 10. PEAK_SURCHARGE_PERCENT, CONCESSION_PERCENT and MIN_BALANCE_PAISE unchanged.
enum class Zone	Yes	Add Airport as the fourth value (ZONE_COUNT = 4).
Expressions	Yes	The fare expression needs a mutually exclusive rule: peak ? +20% : weekend ? −10% : base. Peak status must now also be known together with a weekend flag.
switch statement in fare()	Yes	New case Zone::Airport is needed; if it is forgotten the compiler gives only a warning and the fare becomes 0 (demonstrated below).
SmartCard class	Yes (small)	Must accept a pass: a pass journey costs nothing and must be allowed even when the balance is below Rs 50, so canTravel(0) is not suitable.
New class	Yes	Pass: the two covered zones and a validity period (from day, to day).
Journey data	Yes	A journey needs weekend, concession and day fields; the old signature fare(Zone, bool) is not enough.
Peak check, input/output, tests	Peak check no; others yes	isPeak() unchanged; tests must be extended for weekend, Airport and pass cases.

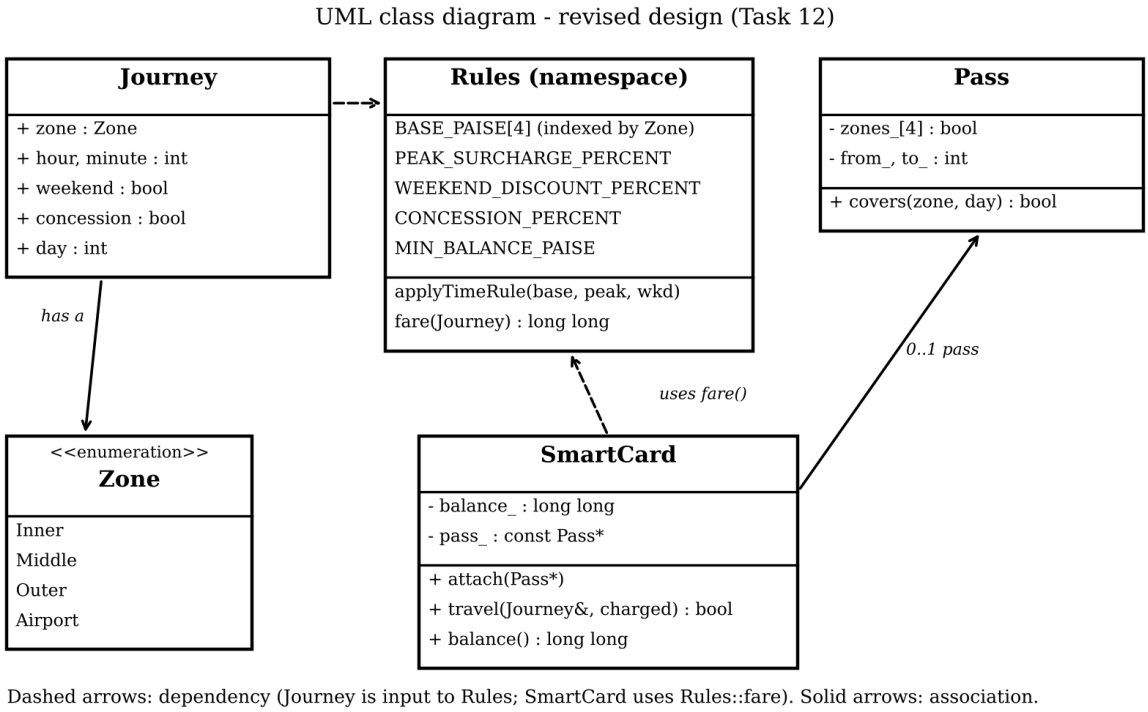


Figure 12.1: UML class diagram of the revised design

B. Evaluation of the Original Design (2 Marks)

Worked example 1 – new zone. The original fare(Zone, bool) is a switch over three zones. Adding Airport to the enum without a new case compiles, but gives a wrong fare:

File: Task12 / impact_demo.cpp

```
// Task 12-A/B evidence: adding Airport to the ORIGINAL design (enum + array + switch)
#include <iostream>
enum class Zone { Inner, Middle, Outer, Airport };          // edit 1: enum value added
constexpr int BASE_PAISE[] = {1000, 2000, 3000};            // edit 2 forgotten: no 4th
entry!
long long fare(Zone z, bool peak) {
   long long base = 0;
   switch (z) {                                             // edit 3 forgotten: no case for Airport
      case Zone::Inner:  base = BASE_PAISE[0]; break;
      case Zone::Middle: base = BASE_PAISE[1]; break;
      case Zone::Outer:  base = BASE_PAISE[2]; break;
   }
   if (peak) base += base * 20 / 100;
   return base;
}
int main() { std::cout << "Airport fare from old design = " << fare(Zone::Airport, false) <<
" paise (should be 6000)\n"; }
```

Output (Screenshot 12): the compiler issues warning: enumeration value ‘Airport’ not handled in switch [-Wswitch], and the program prints Airport fare from old design = 0 paise

(should be 6000): the Airport would be free. Three separate edits are needed (enum, array, switch) and the compiler enforces none of them.

Worked example 2 – weekend discount. Outer, Saturday 12:00 should cost 3000 − 10% = 2700 paise. The old function fare(Zone, bool peak) has no way to receive the weekend information; adding a third parameter changes every caller. Peak and weekend must also be mutually exclusive, which the old single if (peak) expression does not express.

Worked example 3 – monthly pass. A passenger with a monthly pass for Inner and Middle has only Rs 55 on the card and takes an Inner journey. With the old SmartCard, canTravel(1000) tests 5500 − 1000 = 4500 < 5000 and refuses the journey, although the pass promises unlimited journeys. A pass journey has fare 0 and must bypass the balance check, which the original SmartCard interface cannot express, so SmartCard must be modified.

Conclusion of the evaluation: the concepts of the original design (integer paise, constexpr constants, enum class, a SmartCard that guards its balance) absorb the change well, but the switch-based fare(Zone, bool) function and the SmartCard interface cannot absorb it unchanged. A full redesign is not needed; the design is retained where it works and **modified** where it cannot cope.

C. Adaptation or Redesign (2 Marks)

- **Fare table indexed by zone:** constexpr std::array<long long, ZONE_COUNT> BASE_PAISE replaces the switch. Adding a zone is one enum value and one array entry, and the size is tied to ZONE_COUNT; a static_assert checks that the last entry is non-zero so a forgotten entry is a compile error.
- **Separate discount rule:** the function applyTimeRule(base, peak, weekend) states the rule "peak surcharge OR weekend discount, never both" in one place. The concession is applied after it.
- **Pass class:** stores which zones are covered (std::array<bool, 4>) and the validity days; covers(zone, day) tells the card whether the journey is free.
- **SmartCard::travel(journey, charged):** if the attached pass covers the journey, the charge is 0 and the balance rule is skipped; otherwise the normal fare is checked against the Rs 50 minimum balance and deducted.

Pseudocode of the revised design

```
ALGORITHM FareRevised
INPUT:  zone, hour, minute, weekend, hasConcession
OUTPUT: fare in paise
  SET f ← BASE_PAISE[zone]
  IF IsPeak(hour, minute) THEN
    f ← f + f × PEAK_SURCHARGE_PERCENT DIV 100
  ELSE IF weekend THEN
    f ← f − f × WEEKEND_DISCOUNT_PERCENT DIV 100
  ENDIF
  IF hasConcession THEN f ← f − f × CONCESSION_PERCENT DIV 100 ENDIF
  RETURN f
END ALGORITHM
```

```
ALGORITHM Travel
INPUT:  journey, card
OUTPUT: allowed (true or false), charged amount
  IF card has a pass AND pass covers (journey.zone, journey.day) THEN
     charged ← 0 ; RETURN true
  ENDIF
  SET f ← FareRevised(journey)
  IF balance − f < MIN_BALANCE_PAISE THEN RETURN false
  balance ← balance − f ; charged ← f ; RETURN true
END ALGORITHM
```

File: Task12 / main.cpp

```
// Task 12-C: revised (MODIFIED) design - fare table by zone, separate discount rule, Pass
class
#include <array>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <string>

enum class Zone { Inner = 0, Middle = 1, Outer = 2, Airport = 3 };
constexpr std::size_t ZONE_COUNT = 4;

namespace Rules {
   constexpr std::array<long long, ZONE_COUNT> BASE_PAISE{1000, 2000, 3000,
6000};
   constexpr int PEAK_SURCHARGE_PERCENT = 20;
   constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
   constexpr int CONCESSION_PERCENT = 50;
   constexpr long long MIN_BALANCE_PAISE = 5000;
}
static_assert(Rules::BASE_PAISE[ZONE_COUNT - 1] > 0, "fare table must have an entry
for every zone");
inline bool isPeak(int h, int m) { int t = h * 60 + m; return (t >= 480 && t < 600) || (t >=
1020 && t < 1200); }

struct Journey { Zone zone; int hour, minute; bool weekend; bool concession; int day; };

// Separate rule: peak surcharge and weekend discount NEVER combine (peak wins).
long long applyTimeRule(long long base, bool peak, bool weekend) {
   if (peak) return base + base * Rules::PEAK_SURCHARGE_PERCENT / 100;
   if (weekend) return base - base * Rules::WEEKEND_DISCOUNT_PERCENT / 100;
   return base;
```

```
}
long long fare(const Journey& j) {
   const auto index = static_cast<std::size_t>(j.zone);
   if (index >= ZONE_COUNT || j.hour < 0 || j.hour > 23 || j.minute < 0 || j.minute > 59)
return -1;
   long long f = applyTimeRule(Rules::BASE_PAISE[index], isPeak(j.hour, j.minute),
j.weekend);
   if (j.concession) f -= f * Rules::CONCESSION_PERCENT / 100;
   return f;
}

class Pass {                                    // monthly pass: unlimited in two chosen zones
   std::array<bool, ZONE_COUNT> zones_{};
   int from_, to_;
public:
   Pass(Zone a, Zone b, int fromDay, int toDay) : from_(fromDay), to_(toDay) {
      const auto first = static_cast<std::size_t>(a);
      const auto second = static_cast<std::size_t>(b);
      if (first < ZONE_COUNT && second < ZONE_COUNT && fromDay <= toDay) {
         zones_[first] = true; zones_[second] = true;
      }
   }
   bool covers(Zone z, int day) const {
      const auto index = static_cast<std::size_t>(z);
      return day >= from_ && day <= to_ && index < ZONE_COUNT && zones_[index];
   }
};

class SmartCard {
   long long balance_;
   const Pass* pass_ = nullptr;
public:
   explicit SmartCard(long long b) : balance_(b) {}
   void attach(const Pass* p) { pass_ = p; }
   bool travel(const Journey& j, long long& charged) {
      if (pass_ && pass_->covers(j.zone, j.day)) { charged = 0; return true; }   // pass: free,
no balance check
      long long f = fare(j);
      if (f < 0 || balance_ < Rules::MIN_BALANCE_PAISE || f > balance_ -
Rules::MIN_BALANCE_PAISE) { charged = 0; return false; }
      balance_ -= f; charged = f; return true;
   }
   long long balance() const { return balance_; }
```

```
};

int main() {
   int pass = 0, fail = 0;
   auto check = [&](const std::string& name, long long expect, long long actual) {
      bool ok = expect == actual; (ok ? pass : fail)++;
      std::cout << std::left << std::setw(50) << name << std::right << std::setw(6) << expect
<< std::setw(7) << actual
            << "  " << (ok ? "PASS" : "FAIL") << '\n'; };
   std::cout << std::left << std::setw(50) << "Case" << std::right << std::setw(6) <<
"Expect" << std::setw(7) << "Actual" << "  Result\n"
         << std::string(70, '-') << '\n';
   //                        zone        h   m  weekend conc  day
   check("Airport weekday off-peak 12:00",      6000, fare({Zone::Airport,12,
0,false,false,1}));
   check("Airport weekday peak 18:00 (Rs 72)",  7200, fare({Zone::Airport,18,
0,false,false,1}));
   check("Outer weekend off-peak 12:00 (10% off)", 2700, fare({Zone::Outer,12, 0,true,
false,6}));
   check("Outer weekend PEAK 18:00 (no discount)", 3600, fare({Zone::Outer,18, 0,true,
false,6}));
   check("Weekend boundary 10:00 off-peak Inner",   900, fare({Zone::Inner,10, 0,true,
false,6}));
   check("Weekend boundary 08:00 peak Inner",      1200, fare({Zone::Inner, 8, 0,true,
false,6}));
   check("Student Airport weekend off-peak",       2700, fare({Zone::Airport,12,0,true, true,
6}));
   check("Original 3 zones unchanged: Middle peak", 2400,
fare({Zone::Middle,9,0,false,false,1}));

   Pass monthly(Zone::Inner, Zone::Middle, 1, 30);
   SmartCard card(5500);                          // only Rs 55: normal Outer fare would be
denied
   card.attach(&monthly);
   long long c = 0;
   check("Pass: Inner journey allowed (charged 0)", 1,
card.travel({Zone::Inner,9,0,false,false,5}, c));
   check("Pass: charged amount", 0, c);
   check("Pass: Middle peak allowed on low balance", 1,
card.travel({Zone::Middle,18,0,false,false,6}, c));
   check("Pass: Outer not covered, Rs 30 leaves Rs 25 -> denied", 0,
card.travel({Zone::Outer,12,0,false,false,7}, c));
   check("Pass expired on day 31 -> Inner denied", 0,
```

```
card.travel({Zone::Inner,12,0,false,false,31}, c));
    check("Balance untouched by pass journeys", 5500, card.balance());
    std::cout << std::string(70, '-') << "\nPassed " << pass << ", failed " << fail << '\n';

    // timing: switch-style vs table-style lookup is irrelevant to correctness; measure the final
fare()
    auto t0 = std::chrono::steady_clock::now();
    volatile long long sink = 0;
    const long long N = 20000000;
    for (long long i = 0; i < N; ++i) sink = sink + fare({static_cast<Zone>(i & 3),
static_cast<int>(i % 24), 0, (i & 7) == 0, false, 1});
    auto t1 = std::chrono::steady_clock::now();
    std::cout << "fare() cost: " << std::fixed << std::setprecision(2)
              << std::chrono::duration<double, std::nano>(t1 - t0).count() / N << " ns per
journey\n";
    return fail == 0 ? 0 : 1;
}
```

Terminal - Task 12 (g++ 13, Ubuntu)

```
$ g++ -std=c++17 -Wall -Wextra impact_demo.cpp -o impact_demo
impact_demo.cpp: In function 'long long int fare(Zone, bool)':
impact_demo.cpp:7:12: warning: enumeration value 'Airport' not handled in switch [-Wswitch]
    7 |     switch (z) {                                          // edit 3 forgotten: no case for Airport
      |            ^
$ ./impact_demo
Airport fare from old design = 0 paise (should be 6000)
$ g++ -std=c++17 -Wall -Wextra main.cpp -o main
$ ./main
Case                                              Expect Actual  Result
------------------------------------------------------------------------
Airport weekday off-peak 12:00                      6000   6000  PASS
Airport weekday peak 18:00 (Rs 72)                  7200   7200  PASS
Outer weekend off-peak 12:00 (10% off)              2700   2700  PASS
Outer weekend PEAK 18:00 (no discount)              3600   3600  PASS
Weekend boundary 10:00 off-peak Inner                900    900  PASS
Weekend boundary 08:00 peak Inner                   1200   1200  PASS
Student Airport weekend off-peak                    2700   2700  PASS
Original 3 zones unchanged: Middle peak             2400   2400  PASS
Pass: Inner journey allowed (charged 0)                1      1  PASS
Pass: charged amount                                   0      0  PASS
Pass: Middle peak allowed on low balance               1      1  PASS
Pass: Outer not covered, Rs 30 leaves Rs 25 -> denied     0      0  PASS
Pass expired on day 31 -> Inner denied                 0      0  PASS
Balance untouched by pass journeys                  5500   5500  PASS
------------------------------------------------------------------------
Passed 14, failed 0
fare() cost: 6.54 ns per journey
```

Screenshot 12: compile and run of Task12 (impact demonstration and the revised design with 14 tests)

Test table for the revised design (all PASS)

Test	Expected (paise)	Working
Airport weekday off-peak 12:00	6000	Base fare of the new zone

Test	Expected (paise)	Working
Airport weekday peak 18:00	7200	6000 + 20%
Outer weekend off-peak 12:00	2700	3000 − 10%
Outer weekend peak 18:00	3600	Peak wins; no discount: 3000 + 20%
Inner weekend 10:00 (boundary)	900	Off-peak at 10:00, so the discount applies
Inner weekend 08:00 (boundary)	1200	Peak at 08:00, so the surcharge applies and the discount does not
Student Airport weekend off-peak	2700	6000 − 10% = 5400, then 50% concession = 2700
Original zone unchanged: Middle peak	2400	Regression check of an old rule
Pass: Inner journey	allowed, charged 0	Zone covered, day within validity
Pass: Middle peak on Rs 55 balance	allowed	Pass bypasses the Rs 50 rule
Pass: Outer (not covered) on Rs 55	denied	Normal fare Rs 30 would leave Rs 25 < Rs 50
Pass expired (day 31)	denied	Outside the validity period
Balance after pass journeys	5500	Pass journeys do not touch the balance

D. Evidence-Based Defence (2 Marks)

- **Decision: MODIFY.** The tests show that every old rule still gives the same result (Middle peak = 2400, Rs 50 rule, concession) after the change, so retaining the paise/long long, constexpr, enum class and SmartCard foundations was right. Only the parts shown to fail in Part B were changed.
- **Why not RETAIN:** the impact demonstration produced a free Airport journey (fare 0) and the old interfaces cannot carry the weekend flag or the pass; keeping the switch-based function unchanged would give wrong fares.
- **Why not REDESIGN:** all 14 tests of the revised design pass and the 23 tests of Task 9 remain valid for the retained logic; the core classes, money type and constants did not need to be thrown away, so a rewrite would add risk with no benefit.
- **Trade-offs:** (1) the table indexed by zone is simpler to extend than the switch (one entry), but a short table is silently filled with 0 by std::array; the static_assert on the last entry removes this danger. (2) The weekend rule is separated so that it can be audited, at the cost of one more function. (3) The Pass class adds a pointer to SmartCard (8 bytes more per card) and one branch per journey; the measured cost of the complete fare() is about 6.5 ns per journey, i.e. about 20 ms for 3,000,000 journeys, which is negligible. (4) The peak-wins interpretation of “does not combine” is an assumption; if the metro wants the discount to win instead, only the order of the two branches in applyTimeRule() changes.

- **Final decision:** the modified design is correct (14/14 tests), exact (integer paise), scalable (O(1) per journey, 8-byte long long), and every future change is local: one constant for a fare, one enum value plus one table entry for a zone, one rule function for a discount.

Appendix – GitHub Repository, README and Commit Plan

Repository layout

```
DSA01-<RegisterNumber>-Assignment1/
  README.md
  Task01/ main.cpp, output screenshot
  Task02/ main.cpp, output screenshot
  Task03/ main.cpp, output screenshot
  Task04/ predict.cpp, corrected.cpp, narrowing_demo.cpp, screenshot
  Task05/ predict.cpp, expressions.cpp, screenshot
  Task06/ main.cpp, boundary.cpp, array_version.cpp, screenshot
  Task07/ procedural.cpp, oop.cpp, public_member.cpp, screenshot
  Task08/ main.cpp, getline_version.cpp, input_valid.txt, input_invalid.txt,
input_getline.txt, screenshot
  Task09/ main.cpp, table_tests.cpp, screenshot
  Task10/ main.cpp, screenshot
  Task11/ main.cpp, fares.cfg, screenshot
  Task12/ impact_demo.cpp, main.cpp, screenshot
```

README.md

```
# DSA01-<RegisterNumber>-Assignment1 : Namma Metro Smart-Card Fare System

- **Name:** <your name>
- **Register number:** <your register number>
- **Compiler:** g++ (Ubuntu 13.3.0) 13.3.0 (any g++ 11 or later works)
- **Compile command (all tasks):** `g++ -std=c++17 -Wall -Wextra <file>.cpp -o
<name>`

## How to run each program
| Task | Folder | Compile and run | Result |
|------|--------|-----------------|--------|
| 1 | Task01 | `g++ -std=c++17 -Wall -Wextra main.cpp -o main && ./main` | Outer peak
fare 3600 paise, balance 6400, sizeof(SmartCard)=8 |
| 2 | Task02 | same | constexpr vs macro vs enum class demo |
| 3 | Task03 | same | sizes, ranges, double precision error 1.3e-06, int overflow evidence |
| 4 | Task04 | `predict.cpp`, `corrected.cpp`; `narrowing_demo.cpp` is expected to fail to
compile | Output 3 3.5 / C 68 / 1 true / s >= u |
| 5 | Task05 | `predict.cpp`, `expressions.cpp` | 11 16 / 9 6 4 / 6 4 / 3 1 -3 -1 / 18 |
| 6 | Task06 | `main.cpp`, `boundary.cpp`, `array_version.cpp` | Matches the expected
output |
| 7 | Task07 | `procedural.cpp`, `oop.cpp`, `public_member.cpp` | OOP keeps balance at
6400; procedural reaches -800 |
| 8 | Task08 | `./main --echo < input_valid.txt`, `./main --echo < input_invalid.txt`,
`./getline_version < input_getline.txt` | Recovers from bad input |
```

9	Task09	`main.cpp`, `table_tests.cpp`	23 of 23 tests pass
10	Task10	`main.cpp`	Totals for 20k / 500k / 3M journeys, int overflow, timing
11	Task11	`main.cpp` (run inside Task11 so fares.cfg is found)	Final design and config-file timing
12	Task12	`impact_demo.cpp`, `main.cpp`	Old design gives free Airport fare; revised design 14 of 14 tests pass

Commit plan (at least one commit per task)

```
git add Task01 && git commit -m "Task 1: <short description>"
git add Task02 && git commit -m "Task 2: <short description>"
git add Task03 && git commit -m "Task 3: <short description>"
git add Task04 && git commit -m "Task 4: <short description>"
git add Task05 && git commit -m "Task 5: <short description>"
git add Task06 && git commit -m "Task 6: <short description>"
git add Task07 && git commit -m "Task 7: <short description>"
git add Task08 && git commit -m "Task 8: <short description>"
git add Task09 && git commit -m "Task 9: <short description>"
git add Task10 && git commit -m "Task 10: <short description>"
git add Task11 && git commit -m "Task 11: <short description>"
git add Task12 && git commit -m "Task 12: <short description>"
git add README.md && git commit -m "Add README"
git push origin main
```